

In [166...

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Santé du Sommeil & Mode de Vie

Ce projet s'intéresse à l'analyse approfondie des facteurs biologiques et comportementaux (Niveau de stress, Âge, Activité physique, Profession) qui influencent directement la **durée et la qualité du sommeil**.

Après une phase rigoureuse de nettoyage et d'exploration des données (EDA), l'objectif principal est de concevoir et de comparer plusieurs **modèles de Machine Learning (Régression)** afin de prédire avec précision le comportement du sommeil.

 **Travail Réalisé :** Nettoyage Avancé, Analyse Exploratoire (EDA), Visualisation & Modélisation Prédictive (Régression Multi-modèles).

 **Réalisé par :** BAKALI TAHIRI Noha et BOUASSAB Noaman

In [167...

```
df_data = pd.read_csv(r"C:\Users\Noha\OneDrive\Desktop\AI101\Machine Learning\ML\Projet_Regre:
df_data
```

Out[167...

	Person ID	Gender	Age	Occupation	Sleep Duration	Quality of Sleep	Physical Activity Level	Stress Level	Sleep Disorder
0	1.0	Male	NaN	Software Engineer	NaN	6.0	NaN	NaN	NaN
1	20.0	NaN	NaN	Doctor	6.2	-6.0	NaN	NaN	NaN
2	NaN	NaN	NaN	NaN	-6.2	NaN	60.0	8.0	NaN
3	4.0	NaN	-28.0	NaN	NaN	NaN	NaN	NaN	Sleep Apnea
4	NaN	MALE	28.0	NaN	5.9	4.0	30.0	8.0	Seep Apnea
...	...	...	...	...	...	...	...	...	...
369	370.0	NaN	59.0	Nurse	8.1	NaN	NaN	3.0	NaN
370	371.0	Female	NaN	NaN	8.0	9.0	-75.0	3.0	NaN
371	-372.0	Femle	59.0	Nurs	810.0	NaN	NaN	3.0	SLEEP APNEA
372	37300.0	Female	NaN	NaN	NaN	9.0	75.0	3.0	Sleep Apnea
373	NaN	Femle	59.0	Nurse	8.1	NaN	NaN	NaN	SLEEP APNEA

374 rows × 9 columns

EDA (Exploraty Data Analysis) + Data Cleaning

In [168...

```
df = df_data.copy()
```

In [169...

```
df.describe()
```

Out[169...

	Person ID	Age	Sleep Duration	Quality of Sleep	Physical Activity Level	Stress Level
count	222.000000	209.000000	235.000000	210.000000	210.000000	228.000000
mean	2107.864865	442.732057	60.084681	667.414286	71.066667	6.364035
std	7412.071721	1303.522530	194.130721	5838.171253	138.796934	12.706581
min	-33400.000000	-59.000000	-8.500000	-800.000000	-90.000000	-60.000000
25%	77.500000	31.000000	6.100000	6.000000	30.000000	3.000000
50%	206.500000	39.000000	7.100000	8.000000	57.500000	5.000000
75%	331.750000	50.000000	7.800000	9.000000	75.000000	7.000000
max	37300.000000	5400.000000	850.000000	60000.000000	900.000000	80.000000

In [170...

```
df.shape
```

Out[170...

(374, 9)

In [171...

```
df.info()
```

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 374 entries, 0 to 373

Data columns (total 9 columns):

#	Column	Non-Null Count	Dtype
0	Person ID	222 non-null	float64
1	Gender	221 non-null	object
2	Age	209 non-null	float64
3	Occupation	235 non-null	object
4	Sleep Duration	235 non-null	float64
5	Quality of Sleep	210 non-null	float64
6	Physical Activity Level	210 non-null	float64
7	Stress Level	228 non-null	float64
8	Sleep Disorder	101 non-null	object

dtypes: float64(6), object(3)

memory usage: 26.4+ KB

In [172... df.columns

Out[172... Index(['Person ID', 'Gender', 'Age', 'Occupation', 'Sleep Duration',
'Quality of Sleep', 'Physical Activity Level', 'Stress Level',
'Sleep Disorder'],
dtype='object')

In [173... df = df.reset_index(drop=True)
df['Person ID'] = df.index +1

In [174... df

Out[174...

	Person ID	Gender	Age	Occupation	Sleep Duration	Quality of Sleep	Physical Activity Level	Stress Level	Sleep Disorder
	0	1	Male	NaN	Software Engineer	NaN	6.0	NaN	NaN
	1	2	NaN	NaN	Doctor	6.2	-6.0	NaN	NaN
	2	3	NaN	NaN	NaN	-6.2	NaN	60.0	8.0
	3	4	NaN	-28.0	NaN	NaN	NaN	NaN	Sleep Apnea
	4	5	MALE	28.0	NaN	5.9	4.0	30.0	8.0
	...	...	...	...	...	...	...	...	...
	369	370	NaN	59.0	Nurse	8.1	NaN	NaN	3.0
	370	371	Female	NaN	NaN	8.0	9.0	-75.0	3.0
	371	372	Femle	59.0	Nurs	810.0	NaN	NaN	3.0
	372	373	Female	NaN	NaN	NaN	9.0	75.0	3.0
	373	374	Femle	59.0	Nurse	8.1	NaN	NaN	NaN

374 rows × 9 columns

In [175... df['Gender'].unique()

```
Out[175... array(['Male', nan, 'MALE', 'male', 'Mal','ale', 'emale', 'Fmale', 'Mle',
      'Female', 'Femae', 'female', 'Femle', 'FEMALE', 'Femal', 'Feale'],
      dtype=object)
```

```
In [176... df['Gender'] = df['Gender'].astype(str).str[0].str.upper().map({'M': 'Male', 'F': 'Female'})
```

```
In [177... df
```

```
Out[177...
```

	Person ID	Gender	Age	Occupation	Sleep Duration	Quality of Sleep	Physical Activity Level	Stress Level	Sleep Disorder
0	1	Male	NaN	Software Engineer	NaN	6.0	NaN	NaN	NaN
1	2	NaN	NaN	Doctor	6.2	-6.0	NaN	NaN	NaN
2	3	NaN	NaN	NaN	-6.2	NaN	60.0	8.0	NaN
3	4	NaN	-28.0	NaN	NaN	NaN	NaN	NaN	Sleep Apnea
4	5	Male	28.0	NaN	5.9	4.0	30.0	8.0	Seep Apnea
...	...	...	...	...	...	...	...	...	...
369	370	NaN	59.0	Nurse	8.1	NaN	NaN	3.0	NaN
370	371	Female	NaN	NaN	8.0	9.0	-75.0	3.0	NaN
371	372	Female	59.0	Nurs	810.0	NaN	NaN	3.0	SLEEP APNEA
372	373	Female	NaN	NaN	NaN	9.0	75.0	3.0	Sleep Apnea
373	374	Female	59.0	Nurse	8.1	NaN	NaN	NaN	SLEEP APNEA

374 rows × 9 columns

```
In [178... df['Gender'].value_counts()
```

```
Out[178... Gender
Male      123
Female     89
Name: count, dtype: int64
```

```
In [179... df
```

Out[179...

	Person ID	Gender	Age	Occupation	Sleep Duration	Quality of Sleep	Physical Activity Level	Stress Level	Sleep Disorder
0	1	Male	NaN	Software Engineer	NaN	6.0	NaN	NaN	NaN
1	2	NaN	NaN	Doctor	6.2	-6.0	NaN	NaN	NaN
2	3	NaN	NaN	NaN	-6.2	NaN	60.0	8.0	NaN
3	4	NaN	-28.0	NaN	NaN	NaN	NaN	NaN	Sleep Apnea
4	5	Male	28.0	NaN	5.9	4.0	30.0	8.0	Seep Apnea
...	...	...	...	...	...	...	...	...	...
369	370	NaN	59.0	Nurse	8.1	NaN	NaN	3.0	NaN
370	371	Female	NaN	NaN	8.0	9.0	-75.0	3.0	NaN
371	372	Female	59.0	Nurs	810.0	NaN	NaN	3.0	SLEEP APNEA
372	373	Female	NaN	NaN	NaN	9.0	75.0	3.0	Sleep Apnea
373	374	Female	59.0	Nurse	8.1	NaN	NaN	NaN	SLEEP APNEA

374 rows × 9 columns

In [180...

```
df['Gender'].fillna('Male',inplace = True)
```

In [181...

```
df
```

Out[181...

	Person ID	Gender	Age	Occupation	Sleep Duration	Quality of Sleep	Physical Activity Level	Stress Level	Sleep Disorder
0	1	Male	NaN	Software Engineer	NaN	6.0	NaN	NaN	NaN
1	2	Male	NaN	Doctor	6.2	-6.0	NaN	NaN	NaN
2	3	Male	NaN	NaN	-6.2	NaN	60.0	8.0	NaN
3	4	Male	-28.0	NaN	NaN	NaN	NaN	NaN	Sleep Apnea
4	5	Male	28.0	NaN	5.9	4.0	30.0	8.0	Seep Apnea
...	...	...	...	...	...	...	...	...	...
369	370	Male	59.0	Nurse	8.1	NaN	NaN	3.0	NaN
370	371	Female	NaN	NaN	8.0	9.0	-75.0	3.0	NaN
371	372	Female	59.0	Nurs	810.0	NaN	NaN	3.0	SLEEP APNEA
372	373	Female	NaN	NaN	NaN	9.0	75.0	3.0	Sleep Apnea
373	374	Female	59.0	Nurse	8.1	NaN	NaN	NaN	SLEEP APNEA

374 rows × 9 columns

In [182...

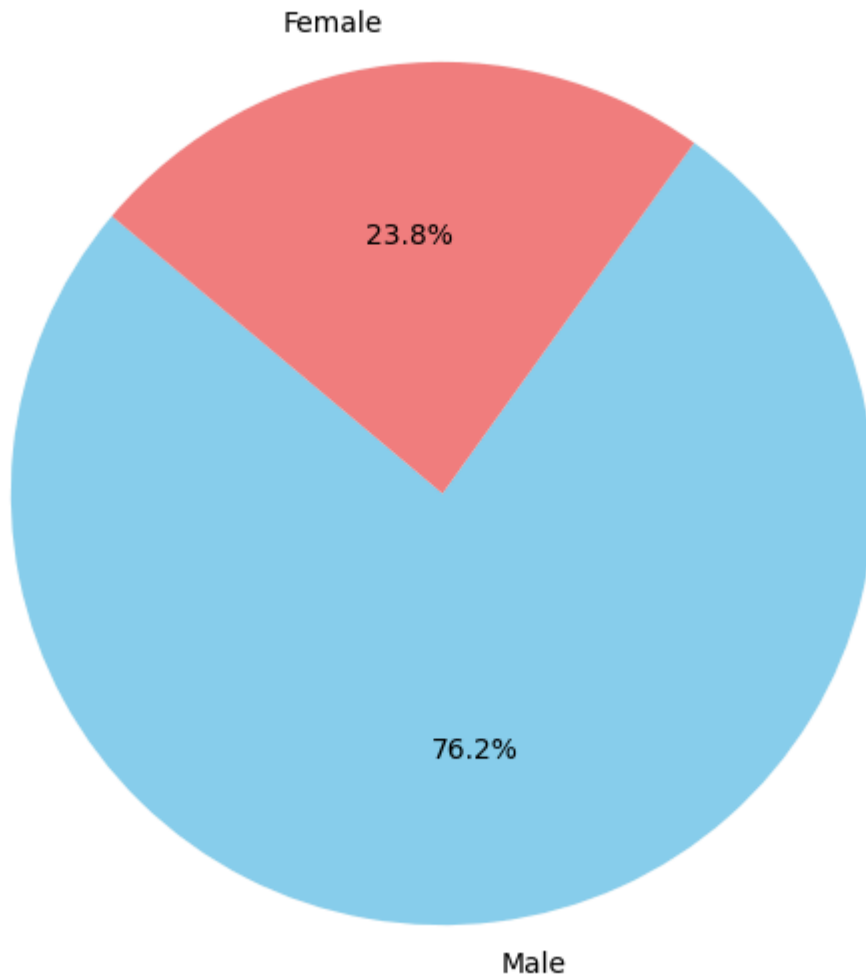
```
gender_counts = df['Gender'].value_counts()

plt.figure(figsize=(7, 7))
plt.pie(gender_counts, labels=gender_counts.index, autopct='%1.1f%%',
        colors=['skyblue', 'lightcoral'], startangle=140)

plt.title("Pourcentage par Genre", fontsize=14)

plt.show()
```

Pourcentage par Genre



Observation de la Répartition

D'après le graphique, nous constatons une **prédominance masculine** dans les données traitées :

- **Hommes (Male)** : Représentent la grande majorité avec **76.2%**.
- **Femmes (Female)** : Représentent **23.8%** de l'échantillon.

In [183... `df.head()`

Out[183...

	Person ID	Gender	Age	Occupation	Sleep Duration	Quality of Sleep	Physical Activity Level	Stress Level	Sleep Disorder
0	1	Male	NaN	Software Engineer	NaN	6.0	NaN	NaN	NaN
1	2	Male	NaN	Doctor	6.2	-6.0	NaN	NaN	NaN
2	3	Male	NaN	NaN	-6.2	NaN	60.0	8.0	NaN
3	4	Male	-28.0	NaN	NaN	NaN	NaN	NaN	Sleep Apnea
4	5	Male	28.0	NaN	5.9	4.0	30.0	8.0	Seep Apnea

```
In [184...] df['Age'].unique()
```

```
Out[184...] array([ nan, -28.,  28.,  29., -29., -30.,  30.,  31., 3100.,
        32., -32.,  33.,  34.,  35., -35.,  36., -36., 3700.,
        37., -37.,  38., 3800., -38.,  39., 3900.,  40., 4000.,
       4100.,  41., -42.,  42., -43., 4300.,  43.,  44., -44.,
       4400., -45., 4500.,  45., -48., -49.,  49., 4900.,  50.,
        -50., 5000.,  51.,  52., 5200.,  53., -53., 5300.,  54.,
       5400.,  55., -56.,  57., -57.,  58., -58.,  59., -59.] )
```

```
In [185...] df['Age'] = df['Age'].abs()
```

```
In [186...] df['Age'].unique()
```

```
Out[186...] array([ nan,  28.,  29.,  30.,  31., 3100.,  32.,  33.,  34.,
        35.,  36., 3700.,  37.,  38., 3800.,  39., 3900.,  40.,
       4000., 4100.,  41.,  42.,  43., 4300.,  44., 4400.,  45.,
       4500.,  48.,  49., 4900.,  50., 5000.,  51.,  52., 5200.,
        53., 5300.,  54., 5400.,  55.,  56.,  57.,  58.,  59.] )
```

```
In [187...] df['Age'] = pd.to_numeric(df['Age'], errors='coerce')
```

```
In [188...] def corriger_age(age):
              if age > 120:
                  return age / 100
              else:
                  return age

df['Age'] = df['Age'].apply(corriger_age)
```

```
In [189...] df['Age'].unique()
```

```
Out[189...] array([nan, 28., 29., 30., 31., 32., 33., 34., 35., 36., 37., 38., 39.,
        40., 41., 42., 43., 44., 45., 48., 49., 50., 51., 52., 53., 54.,
        55., 56., 57., 58., 59.] )
```

```
In [190...] df['Age'].isnull().sum()
```

```
Out[190...] np.int64(165)
```

```
In [191...] df.describe()
```

	Person ID	Age	Sleep Duration	Quality of Sleep	Physical Activity Level	Stress Level
count	374.000000	209.000000	235.000000	210.000000	210.000000	228.000000
mean	187.500000	41.923445	60.084681	667.414286	71.066667	6.364035
std	108.108742	8.791598	194.130721	5838.171253	138.796934	12.706581
min	1.000000	28.000000	-8.500000	-800.000000	-90.000000	-60.000000
25%	94.250000	35.000000	6.100000	6.000000	30.000000	3.000000
50%	187.500000	41.000000	7.100000	8.000000	57.500000	5.000000
75%	280.750000	49.000000	7.800000	9.000000	75.000000	7.000000
max	374.000000	59.000000	850.000000	60000.000000	900.000000	80.000000

```
In [192...] df['Age'] = df['Age'].fillna(df['Age'].mean())
```

```
In [193...] df['Age'].isnull().sum()
```

Out[193... np.int64(0)

In [194... df.info()

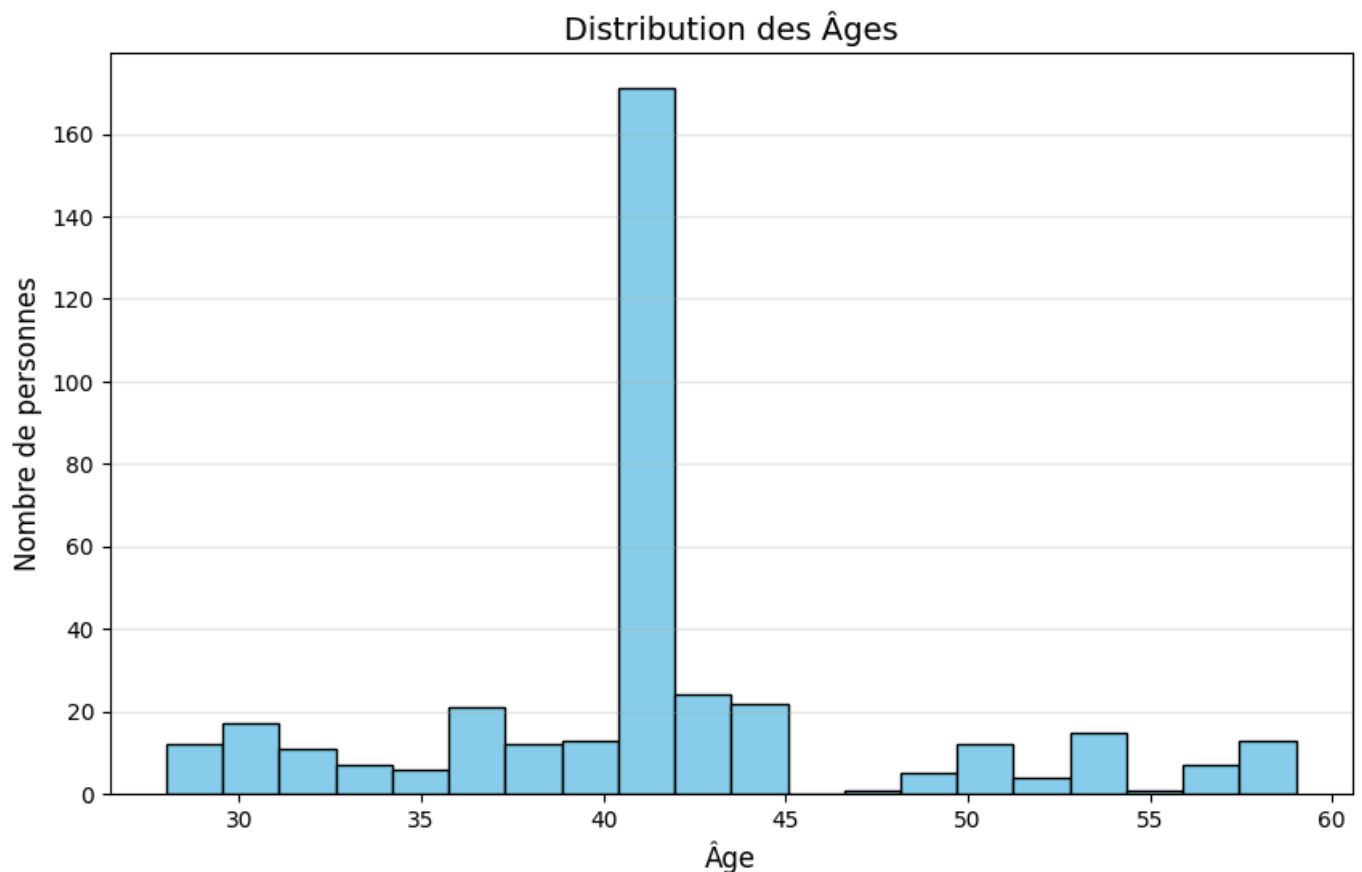
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 374 entries, 0 to 373
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Person ID             374 non-null   int64
1   Gender                 374 non-null   object
2   Age                    374 non-null   float64
3   Occupation             235 non-null   object
4   Sleep Duration         235 non-null   float64
5   Quality of Sleep       210 non-null   float64
6   Physical Activity Level 210 non-null   float64
7   Stress Level           228 non-null   float64
8   Sleep Disorder         101 non-null   object
dtypes: float64(5), int64(1), object(3)
memory usage: 26.4+ KB
```

```
In [195... plt.figure(figsize=(10, 6))

plt.hist(df['Age'], bins=20, color='skyblue', edgecolor='black')

plt.title("Distribution des Âges", fontsize=14)
plt.xlabel("Âge", fontsize=12)
plt.ylabel("Nombre de personnes", fontsize=12)

plt.grid(axis='y', alpha=0.3)
plt.show()
```



Distribution des Âges

Analyse de la pyramide des âges de l'échantillon :

- **Point Majeur :** Une concentration massive de données autour de **42-43 ans**.
- **Diversité :** Présence de données allant de 25 à 60 ans, mais de façon moins dense.

Note : La forte présence de la quarantaine suggère que les conclusions sur le sommeil seront particulièrement précises pour cette tranche d'âge.

```
In [196... df['Age'].isnull().sum()
```

```
Out[196... np.int64(0)
```

```
In [197... df['Age'] = df['Age'].round(2)
```

```
In [198... df['Age'].value_counts()
```

```
Out[198... Age
41.92    165
43.00     18
44.00     14
37.00     13
38.00     12
32.00     11
31.00     10
53.00     10
50.00     10
29.00      9
59.00      9
39.00      9
36.00      8
45.00      8
30.00      7
33.00      6
41.00      6
42.00      6
35.00      6
57.00      6
49.00      5
54.00      5
40.00      4
58.00      4
52.00      4
28.00      3
51.00      2
34.00      1
48.00      1
55.00      1
56.00      1
Name: count, dtype: int64
```

```
In [199... df
```

Out[199...

	Person ID	Gender	Age	Occupation	Sleep Duration	Quality of Sleep	Physical Activity Level	Stress Level	Sleep Disorder
0	1	Male	41.92	Software Engineer	NaN	6.0	NaN	NaN	NaN
1	2	Male	41.92	Doctor	6.2	-6.0	NaN	NaN	NaN
2	3	Male	41.92	NaN	-6.2	NaN	60.0	8.0	NaN
3	4	Male	28.00	NaN	NaN	NaN	NaN	NaN	Sleep Apnea
4	5	Male	28.00	NaN	5.9	4.0	30.0	8.0	Seep Apnea
...	...	...	...	...	...	...	...	...	...
369	370	Male	59.00	Nurse	8.1	NaN	NaN	3.0	NaN
370	371	Female	41.92	NaN	8.0	9.0	-75.0	3.0	NaN
371	372	Female	59.00	Nurs	810.0	NaN	NaN	3.0	SLEEP APNEA
372	373	Female	41.92	NaN	NaN	9.0	75.0	3.0	Sleep Apnea
373	374	Female	59.00	Nurse	8.1	NaN	NaN	NaN	SLEEP APNEA

374 rows × 9 columns

In [200...

```
df['Occupation'].unique()
```

Out[200...

```
array(['Software Engineer', 'Doctor', nan, 'TEACHER', 'doctor', 'Nurse',
      'DOCTOR', 'Dotor', 'octor', 'Scientist', 'Teacher',
      'SOFTWARE ENGINEER', 'Engineer', 'ENGINEER', 'LAWYER',
      'accountant', 'Accountant', 'teacher', 'ACCOUNTANT', 'Lawyer',
      'Lwyer', 'lawyer', 'Lawer', 'Salesperson', 'SALESPERSON',
      'Saleperson', 'engineer', 'salesperson', 'Manager', 'Nuse',
      'nurse', 'Engiee', 'Enginee', 'Nurs', 'NURSE'], dtype=object)
```

In [201...

```
def clean_occupation(x):
    x = str(x).strip()

    if x in ['nurse', 'Nuse', 'NURSE', 'Nurs', 'Nurse']:
        return 'Nurse'

    elif x in ['Lwyer', 'Lawer', 'lawyer', 'LAWYER', 'Lawyer']:
        return 'Lawyer'

    elif x in ['DOCTOR', 'doctor', 'Dotor', 'octor', 'Doctor']:
        return 'Doctor'

    elif x in ['engineer', 'ENGINEER', 'Enginee', 'Engiee', 'Engineer']:
        return 'Engineer'

    elif x in ['ACCOUNTANT', 'accountant']:
        return 'Accountant'

    elif x in ['SOFTWARE ENGINEER']:
        return 'Software Engineer'

    elif x in ['teacher', 'TEACHER']:
        return 'Teacher'
```

```

    elif x in ['salesperson', 'Saleperson', 'SALESPERSON']:
        return 'Salesperson'

    return x

df['Occupation'] = df['Occupation'].apply(clean_occupation)

print(df['Occupation'].unique())

```

```

['Software Engineer' 'Doctor' 'nan' 'Teacher' 'Nurse' 'Scientist'
 'Engineer' 'Lawyer' 'Accountant' 'Salesperson' 'Manager']

```

```

In [202... df = df.replace(['nan', 'NaN', 'NAN', 'n/a'], np.nan)

```

```

In [203... df['Occupation'].value_counts()

```

```

Out[203... Occupation
Nurse                54
Doctor              45
Engineer            34
Lawyer              28
Teacher             24
Accountant          23
Salesperson         20
Scientist           4
Software Engineer   2
Manager             1
Name: count, dtype: int64

```

```

In [204... df['Occupation'].isnull().sum()

```

```

Out[204... np.int64(139)

```

```

In [205... df['Occupation'] = df['Occupation'].fillna('Nurse')

```

```

In [206... df['Occupation'].value_counts()

```

```

Out[206... Occupation
Nurse                193
Doctor              45
Engineer            34
Lawyer              28
Teacher             24
Accountant          23
Salesperson         20
Scientist           4
Software Engineer   2
Manager             1
Name: count, dtype: int64

```

```

In [207... counts = df['Occupation'].value_counts()

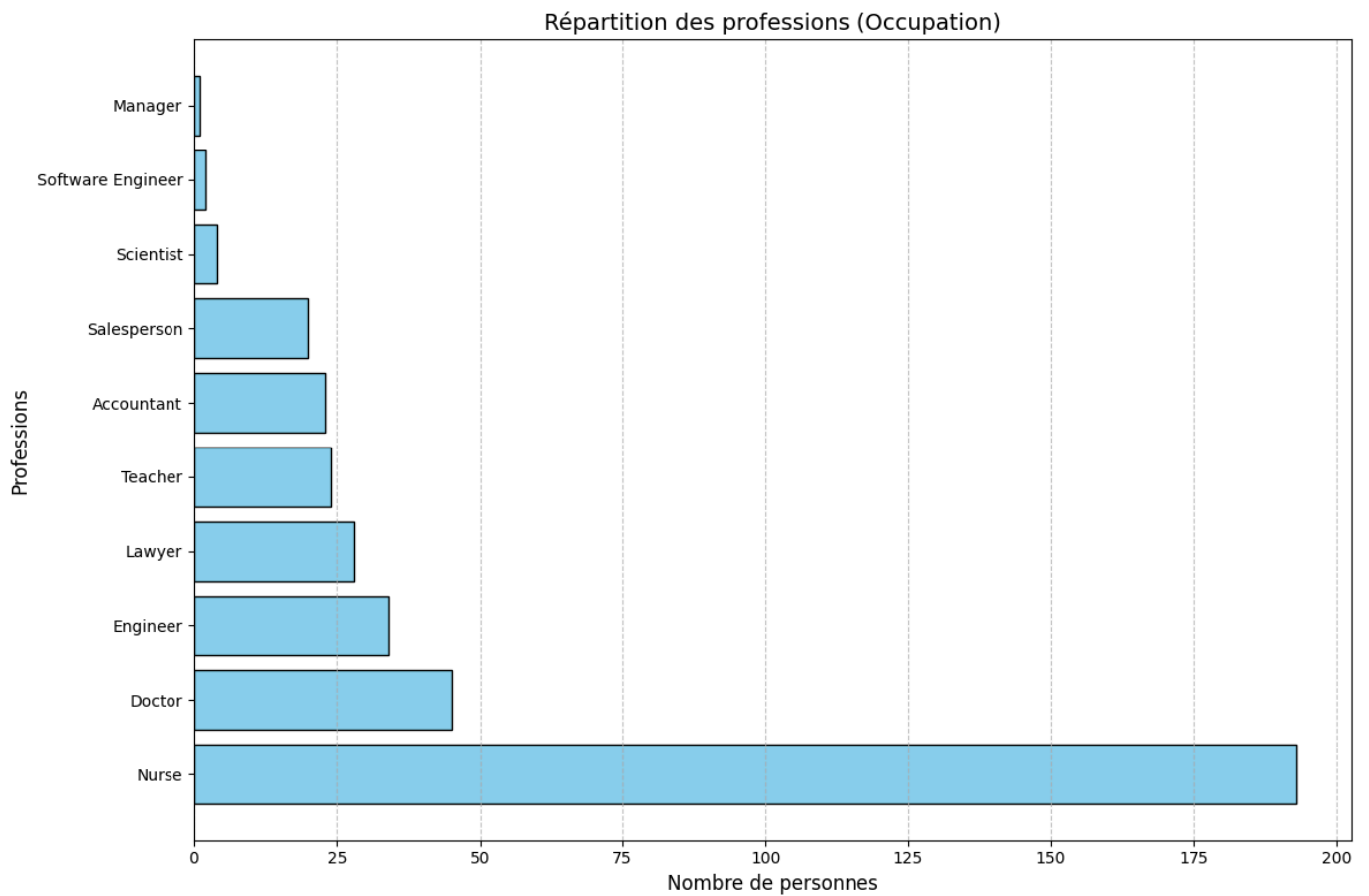
plt.figure(figsize=(12, 8))
plt.barh(counts.index, counts.values, color='skyblue', edgecolor='black')

plt.title("Répartition des professions (Occupation)", fontsize=14)
plt.xlabel("Nombre de personnes", fontsize=12)
plt.ylabel("Professions", fontsize=12)

plt.grid(axis='x', linestyle='--', alpha=0.7)

plt.tight_layout()
plt.show()

```



Répartition par Profession

Observation : La catégorie **Nurse** est prédominante (> 175 personnes), suivie par les Docteurs.

```
In [208... df.describe()
```

	Person ID	Age	Sleep Duration	Quality of Sleep	Physical Activity Level	Stress Level
count	374.000000	374.000000	235.000000	210.000000	210.000000	228.000000
mean	187.500000	41.921925	60.084681	667.414286	71.066667	6.364035
std	108.108742	6.565156	194.130721	5838.171253	138.796934	12.706581
min	1.000000	28.000000	-8.500000	-800.000000	-90.000000	-60.000000
25%	94.250000	39.000000	6.100000	6.000000	30.000000	3.000000
50%	187.500000	41.920000	7.100000	8.000000	57.500000	5.000000
75%	280.750000	43.000000	7.800000	9.000000	75.000000	7.000000
max	374.000000	59.000000	850.000000	60000.000000	900.000000	80.000000

```
In [209... df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 374 entries, 0 to 373
```

```
Data columns (total 9 columns):
```

#	Column	Non-Null Count	Dtype
0	Person ID	374 non-null	int64
1	Gender	374 non-null	object
2	Age	374 non-null	float64
3	Occupation	374 non-null	object
4	Sleep Duration	235 non-null	float64
5	Quality of Sleep	210 non-null	float64
6	Physical Activity Level	210 non-null	float64
7	Stress Level	228 non-null	float64
8	Sleep Disorder	101 non-null	object

```
dtypes: float64(5), int64(1), object(3)
```

```
memory usage: 26.4+ KB
```

```
In [210... df['Sleep Duration'] = df['Sleep Duration'].abs()
```

```
In [211... def corriger_sleep_duration(sleep):  
    if sleep > 24:  
        return sleep / 100  
    else:  
        return sleep  
  
df['Sleep Duration'] = df['Sleep Duration'].apply(corriger_sleep_duration)
```

```
In [212... df['Sleep Duration'].dtype
```

```
Out[212... dtype('float64')
```

```
In [213... df['Sleep Duration'].unique()
```

```
Out[213... array([nan, 6.2, 5.9, 6.3, 7.8, 6.1, 6. , 6.5, 7.6, 7.7, 7.9, 6.4, 7.5,  
        7.2, 5.8, 6.7, 7.3, 7.4, 7.1, 6.6, 8. , 6.8, 8.1, 8.3, 8.5, 8.4,  
        8.2])
```

```
In [214... df['Sleep Duration'].isnull().sum()
```

```
Out[214... np.int64(139)
```

```
In [215... df['Sleep Duration'].mean()
```

```
Out[215... np.float64(7.118723404255319)
```

```
In [216... df['Sleep Duration'] = df['Sleep Duration'].fillna(df['Sleep Duration'].mean())
```

```
In [217... df['Sleep Duration'] = df['Sleep Duration'].round(2)
```

```
In [218... df['Sleep Duration'].value_counts()
```

Out[218... Sleep Duration

7.12	139
7.20	24
6.00	18
6.10	16
7.70	16
7.80	16
6.50	15
6.60	15
7.10	13
7.30	11
6.30	10
8.00	9
8.10	9
8.50	8
7.60	8
8.20	8
6.20	7
8.40	6
6.40	5
6.80	3
7.40	3
7.50	3
6.70	3
8.30	3
5.90	2
7.90	2
5.80	2

Name: count, dtype: int64

```
In [219... import matplotlib.pyplot as plt

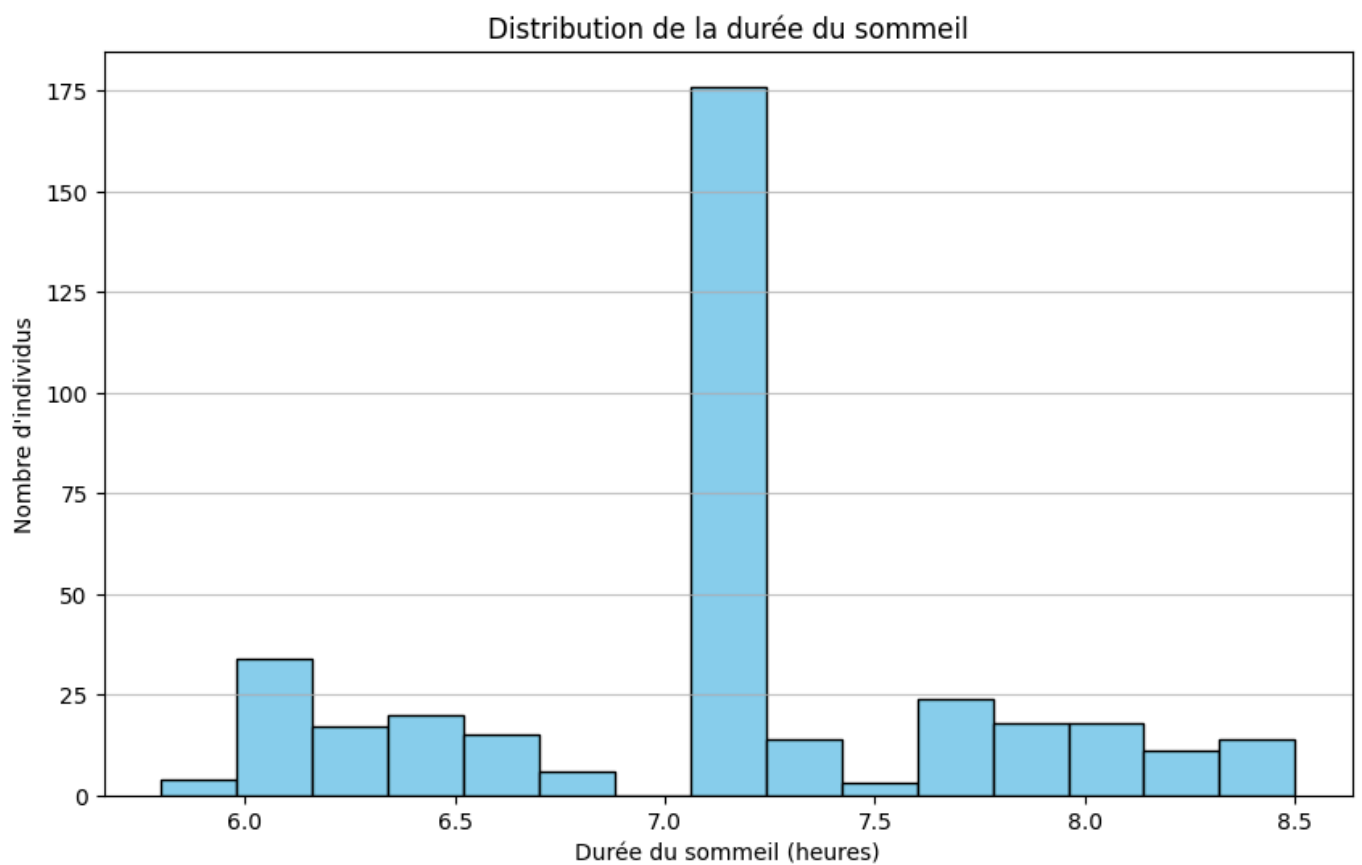
plt.figure(figsize=(10, 6))

plt.hist(df['Sleep Duration'], bins=15, color='skyblue', edgecolor='black')

plt.title("Distribution de la durée du sommeil")
plt.xlabel("Durée du sommeil (heures)")
plt.ylabel("Nombre d'individus")

plt.grid(axis='y', alpha=0.75)

plt.show()
```



Distribution de la Durée du Sommeil

Analyse des heures de repos :

- **Valeur dominante** : La grande majorité des individus dorment environ **7.2 heures**.
- **Variabilité** : La durée varie entre 5.8 et 8.5 heures, mais avec des effectifs plus réduits aux extrémités.

Note : Un sommeil de 7.2h est cliniquement considéré comme une durée saine et équilibrée pour un adulte.

In [220... `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 374 entries, 0 to 373
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Person ID             374 non-null   int64
1   Gender                374 non-null   object
2   Age                   374 non-null   float64
3   Occupation            374 non-null   object
4   Sleep Duration        374 non-null   float64
5   Quality of Sleep      210 non-null   float64
6   Physical Activity Level 210 non-null   float64
7   Stress Level          228 non-null   float64
8   Sleep Disorder        101 non-null   object
dtypes: float64(5), int64(1), object(3)
memory usage: 26.4+ KB
```

In [221... `df['Quality of Sleep'].isnull().value_counts()`

```
Out[221... Quality of Sleep
False      210
True       164
Name: count, dtype: int64
```

```
In [222... df['Quality of Sleep'].nunique()
```

```
Out[222... 18
```

```
In [223... df['Quality of Sleep'].value_counts()
```

```
Out[223... Quality of Sleep
8.0      48
6.0      40
7.0      36
9.0      34
600.0     12
800.0      8
700.0      6
5.0       5
900.0      3
4.0       3
-9.0       3
-7.0       3
-800.0     3
60000.0    2
-6.0       1
500.0       1
-8.0       1
400.0       1
Name: count, dtype: int64
```

```
In [224... df['Quality of Sleep'] = df['Quality of Sleep'].abs()
```

```
In [225... df.loc[df['Quality of Sleep'] > 10, 'Quality of Sleep'] = df['Quality of Sleep'] / 100

while df['Quality of Sleep'].max() > 10:
    df.loc[df['Quality of Sleep'] > 10, 'Quality of Sleep'] = df['Quality of Sleep'] / 10
```

```
In [226... df['Quality of Sleep'].value_counts()
```

```
Out[226... Quality of Sleep
8.0      60
6.0      55
7.0      45
9.0      40
5.0       6
4.0       4
Name: count, dtype: int64
```

```
In [227... df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 374 entries, 0 to 373
```

```
Data columns (total 9 columns):
```

#	Column	Non-Null Count	Dtype
0	Person ID	374 non-null	int64
1	Gender	374 non-null	object
2	Age	374 non-null	float64
3	Occupation	374 non-null	object
4	Sleep Duration	374 non-null	float64
5	Quality of Sleep	210 non-null	float64
6	Physical Activity Level	210 non-null	float64
7	Stress Level	228 non-null	float64
8	Sleep Disorder	101 non-null	object

```
dtypes: float64(5), int64(1), object(3)
```

```
memory usage: 26.4+ KB
```

```
In [228... df['Quality of Sleep'] = df['Quality of Sleep'].fillna(df['Quality of Sleep'].mean())
```

```
In [229... df['Physical Activity Level'].isnull().value_counts()
```

```
Out[229... Physical Activity Level
False      210
True       164
Name: count, dtype: int64
```

```
In [230... df['Physical Activity Level'].value_counts()
```

```
Out[230... Physical Activity Level
30.0        35
45.0        33
60.0        32
90.0        31
75.0        26
-75.0         7
-60.0         6
-90.0         5
40.0         4
-30.0         3
-45.0         3
450.0         3
85.0         2
35.0         2
50.0         2
55.0         2
900.0         2
600.0         2
80.0         2
750.0         1
700.0         1
70.0         1
42.0         1
47.0         1
550.0         1
65.0         1
-55.0         1
Name: count, dtype: int64
```

```
In [231... df['Physical Activity Level'] = df['Physical Activity Level'].abs()
```

```
In [232... df.loc[df['Physical Activity Level'] > 100, 'Physical Activity Level'] = df['Physical Activity Level']
```

```
In [233... df['Physical Activity Level'] = df['Physical Activity Level'].fillna(df['Physical Activity Level'].mean())
```

```
In [234...] df['Physical Activity Level'] = df['Physical Activity Level'].round(2)
```

```
In [235...] df['Physical Activity Level'].value_counts()
```

```
Out[235...] Physical Activity Level
59.23      164
60.00       40
45.00       39
30.00       38
90.00       38
75.00       34
55.00        4
40.00        4
50.00        2
35.00        2
80.00        2
70.00        2
85.00        2
42.00        1
47.00        1
65.00        1
Name: count, dtype: int64
```

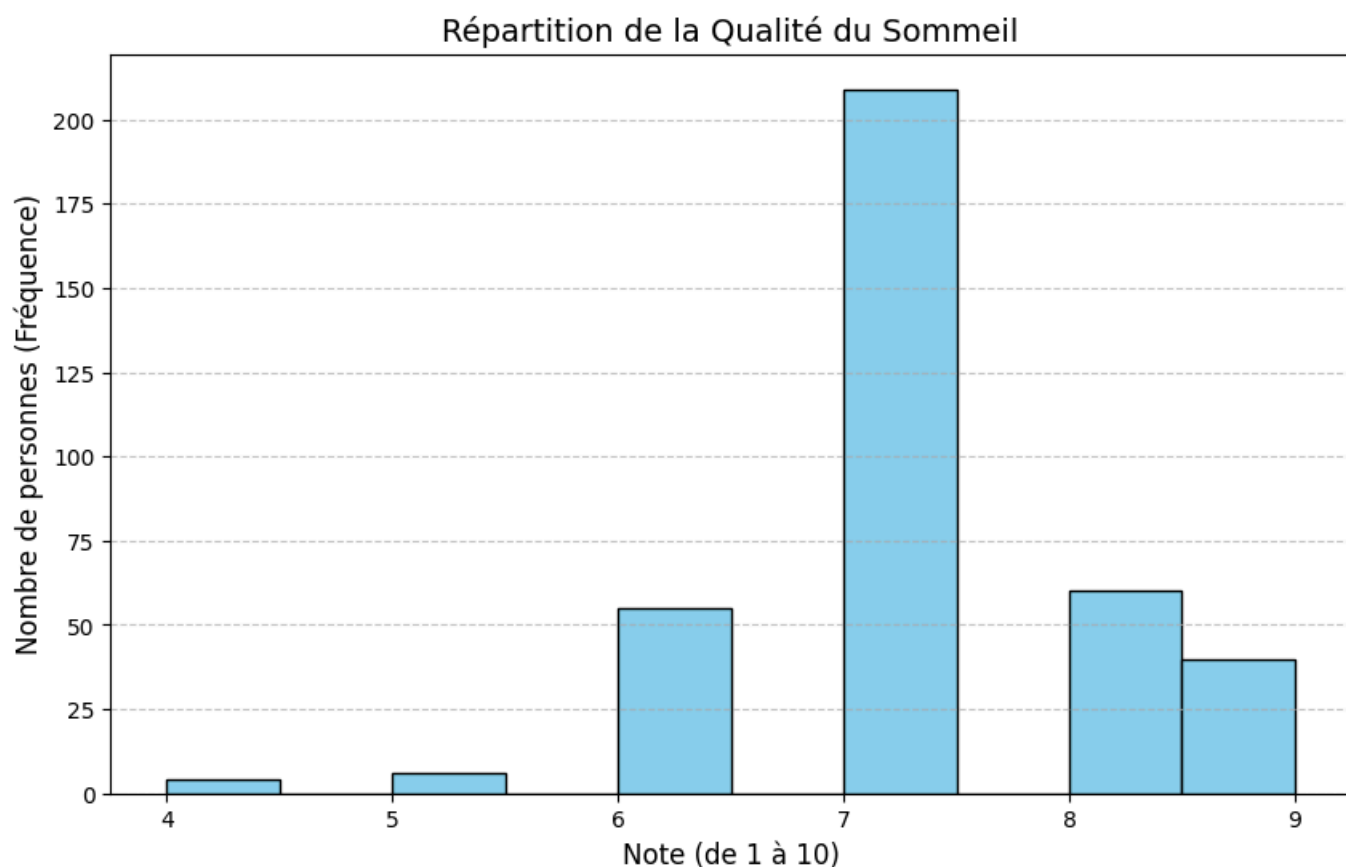
```
In [236...] plt.figure(figsize=(10, 6))

plt.hist(df['Quality of Sleep'], bins=10, color='skyblue', edgecolor='black')

plt.title('Répartition de la Qualité du Sommeil', fontsize=14)
plt.xlabel('Note (de 1 à 10)', fontsize=12)
plt.ylabel('Nombre de personnes (Fréquence)', fontsize=12)

plt.grid(axis='y', linestyle='--', alpha=0.7)

plt.show()
```



- **Durée** : Un pic massif à **7.2 heures**.
- **Qualité** : La note de **7/10** est la plus fréquente.

Note : Globalement, l'échantillon a un sommeil plutôt satisfaisant.

In [237... `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 374 entries, 0 to 373
Data columns (total 9 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Person ID                            374 non-null    int64
1   Gender                               374 non-null    object
2   Age                                   374 non-null    float64
3   Occupation                           374 non-null    object
4   Sleep Duration                       374 non-null    float64
5   Quality of Sleep                     374 non-null    float64
6   Physical Activity Level              374 non-null    float64
7   Stress Level                         228 non-null    float64
8   Sleep Disorder                       101 non-null    object
dtypes: float64(5), int64(1), object(3)
memory usage: 26.4+ KB
```

In [238... `df['Stress Level'].unique()`

Out[238... `array([nan, 8., 6., -8., 7., -60., -6., 4., 60., 3., 80.,
 5., -4., -7., -5., 50., 40., 70., 30., -3.]`

In [239... `df['Stress Level'] = df['Stress Level'].abs()`

In [240... `df.loc[df['Stress Level'] > 10, 'Stress Level'] = df['Stress Level'] / 10`

In [241... `df['Stress Level'] = df['Stress Level'].fillna(df['Stress Level'].mean())`

In [242... `df['Stress Level'] = df['Stress Level'].round(2)`

In [243... `df['Stress Level'].value_counts()`

Out[243... `Stress Level`

5.29	146
3.00	47
4.00	45
5.00	39
8.00	38
7.00	31
6.00	28

Name: count, dtype: int64

In [244... `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 374 entries, 0 to 373
Data columns (total 9 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Person ID                            374 non-null    int64
1   Gender                               374 non-null    object
2   Age                                  374 non-null    float64
3   Occupation                           374 non-null    object
4   Sleep Duration                       374 non-null    float64
5   Quality of Sleep                     374 non-null    float64
6   Physical Activity Level               374 non-null    float64
7   Stress Level                         374 non-null    float64
8   Sleep Disorder                       101 non-null    object
dtypes: float64(5), int64(1), object(3)
memory usage: 26.4+ KB
```

```
In [245... df['Sleep Disorder'].unique()
```

```
Out[245... array([nan, 'Sleep Apnea', 'Seep Apnea', 'Insomnia', 'slee apnea',
        'sleep apnea', 'Insomna', 'Insmnia', 'insomnia', 'INSOMNIA',
        'SEEP APNEA', 'Sleep Anea', 'SLEEP APNEA', 'SleepApnea'],
      dtype=object)
```

```
In [246... df['Sleep Disorder'] = df['Sleep Disorder'].str.strip().str.lower()
```

```
corrections = {
    'insomna': 'Insomnia',
    'insmnia': 'Insomnia',
    'insomnia': 'Insomnia',
    'sleep apnea': 'Sleep Apnea',
    'seep apnea': 'Sleep Apnea',
    'slee apnea': 'Sleep Apnea',
    'sleep anea': 'Sleep Apnea',
    'sleepapnea': 'Sleep Apnea'
}

df['Sleep Disorder'] = df['Sleep Disorder'].replace(corrections)

df['Sleep Disorder'] = df['Sleep Disorder'].fillna('None')

df['Sleep Disorder'].unique()
```

```
Out[246... array(['None', 'Sleep Apnea', 'Insomnia'], dtype=object)
```

```
In [247... df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 374 entries, 0 to 373
Data columns (total 9 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Person ID                            374 non-null    int64
1   Gender                               374 non-null    object
2   Age                                  374 non-null    float64
3   Occupation                           374 non-null    object
4   Sleep Duration                       374 non-null    float64
5   Quality of Sleep                     374 non-null    float64
6   Physical Activity Level               374 non-null    float64
7   Stress Level                         374 non-null    float64
8   Sleep Disorder                       374 non-null    object
dtypes: float64(5), int64(1), object(3)
memory usage: 26.4+ KB
```

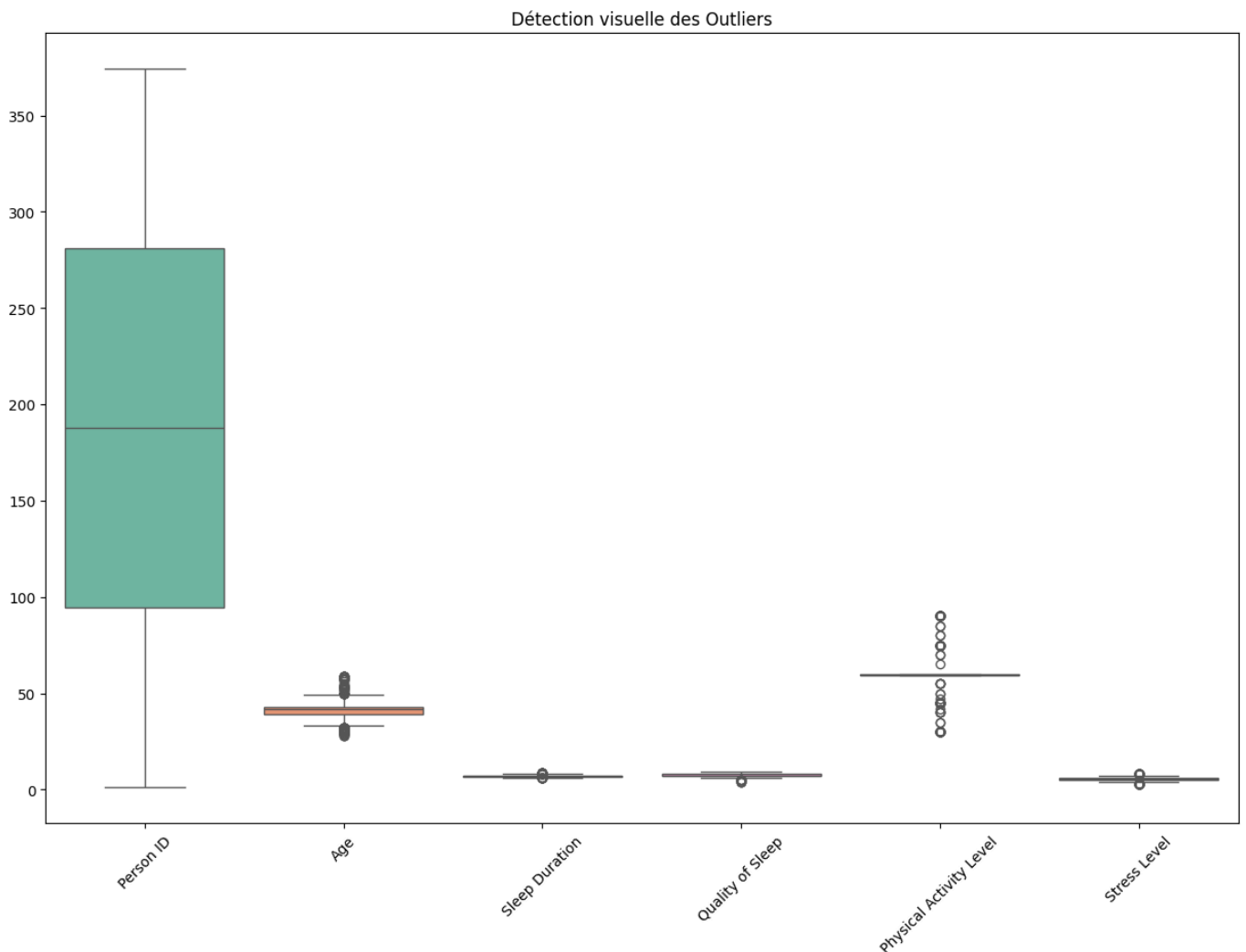
Gestion d'Outliers

In [248...

```
import seaborn as sns

numeric_cols = df.select_dtypes(include=['float64', 'int64']).columns

plt.figure(figsize=(15, 10))
sns.boxplot(data=df[numeric_cols], palette="Set2")
plt.xticks(rotation=45)
plt.title("Détection visuelle des Outliers")
plt.show()
```



Analyse Visuelle des Outliers

L'utilisation des **Boxplots** a permis d'identifier quelques points isolés :

- **Age** : Quelques individus se situent en dehors de la moyenne centrale (42 ans).
- **Activité Physique** : Des pics d'activité intense sont visibles comme des points isolés.
- **Décision** : Ces valeurs sont conservées car elles représentent des cas réels et non des erreurs de saisie.

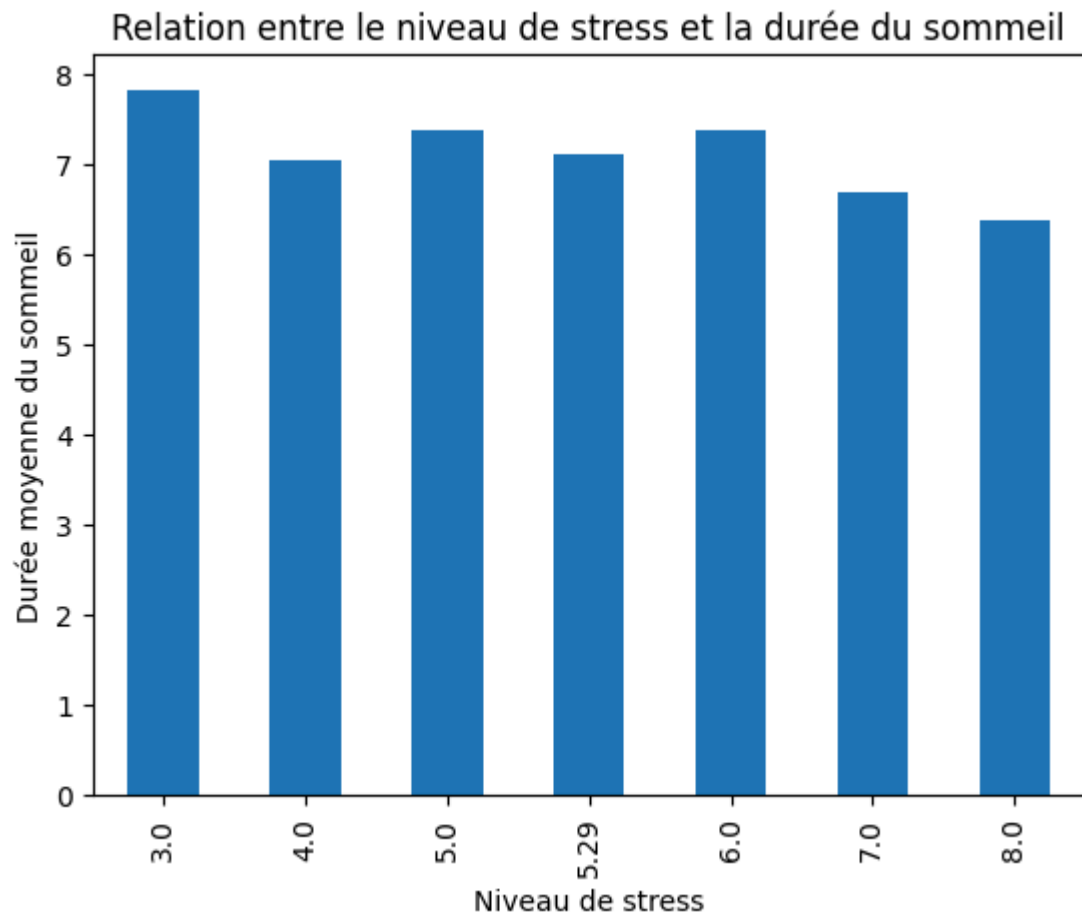
Note : En Data Science Santé, un point isolé n'est pas toujours une erreur, c'est parfois une exception clinique intéressante.

Questions de l'analyse

1-Le stress influence-t-il la durée du sommeil ? (Objectif : Analyser la relation entre le niveau de stress et la durée du sommeil)

In [249...

```
df.groupby('Stress Level')['Sleep Duration'].mean().plot(kind='bar')
plt.title('Relation entre le niveau de stress et la durée du sommeil')
plt.xlabel('Niveau de stress')
plt.ylabel('Durée moyenne du sommeil')
plt.show()
```



Stress vs Durée du Sommeil

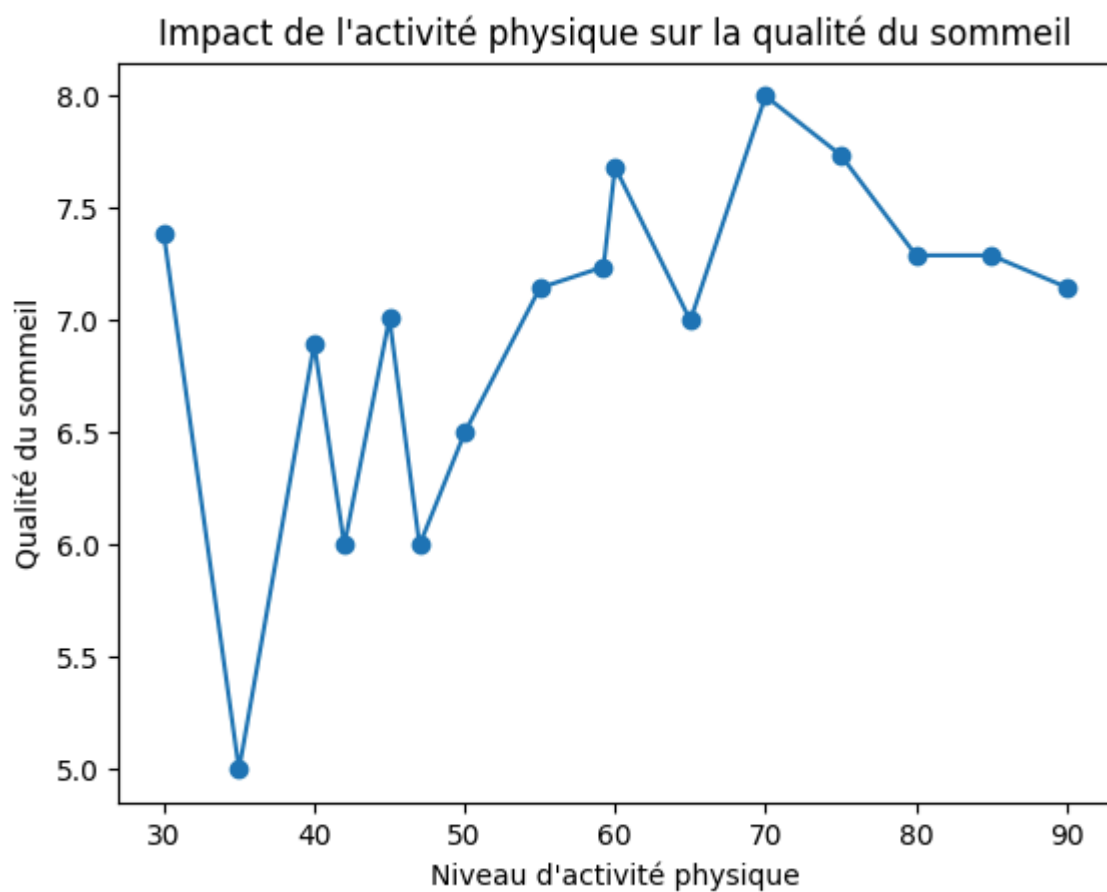
Corrélation : On remarque que les personnes avec un **Stress Level de 8.0** dorment moins (~6.2h) que celles à bas stress.

Conclusion : *Le stress est un facteur limitant direct de la quantité de repos.*

2-L'activité physique améliore-t-elle la qualité du sommeil ? (Objectif : Étudier l'impact de l'activité physique sur la qualité du sommeil.)

In [250...

```
df.groupby('Physical Activity Level')['Quality of Sleep'].mean().plot(kind='line', marker='o')
plt.title("Impact de l'activité physique sur la qualité du sommeil")
plt.xlabel("Niveau d'activité physique")
plt.ylabel("Qualité du sommeil")
plt.show()
```



Niveau d'Activité Physique

Distribution : Les données sont concentrées sur des niveaux précis (30, 60, 90), suggérant des paliers d'exercice.

In [251...

```
check_35 = df[(df['Age'] >= 34) & (df['Age'] <= 36)]  
check_35[['Age', 'Quality of Sleep', 'Occupation']]
```

Out[251...

	Age	Quality of Sleep	Occupation
80	34.0	4.000000	Scientist
82	35.0	7.290476	Nurse
86	35.0	8.000000	Nurse
88	35.0	8.000000	Engineer
90	35.0	7.290476	Nurse
91	35.0	8.000000	Nurse
92	35.0	7.290476	Nurse
95	36.0	8.000000	Accountant
96	36.0	8.000000	Nurse
97	36.0	8.000000	Nurse
99	36.0	7.290476	Teacher
100	36.0	8.000000	Teacher
102	36.0	8.000000	Nurse
103	36.0	5.000000	Nurse
105	36.0	5.000000	Nurse

Activité Physique vs Qualité du Sommeil

Étude de l'impact du sport sur le repos nocturne :

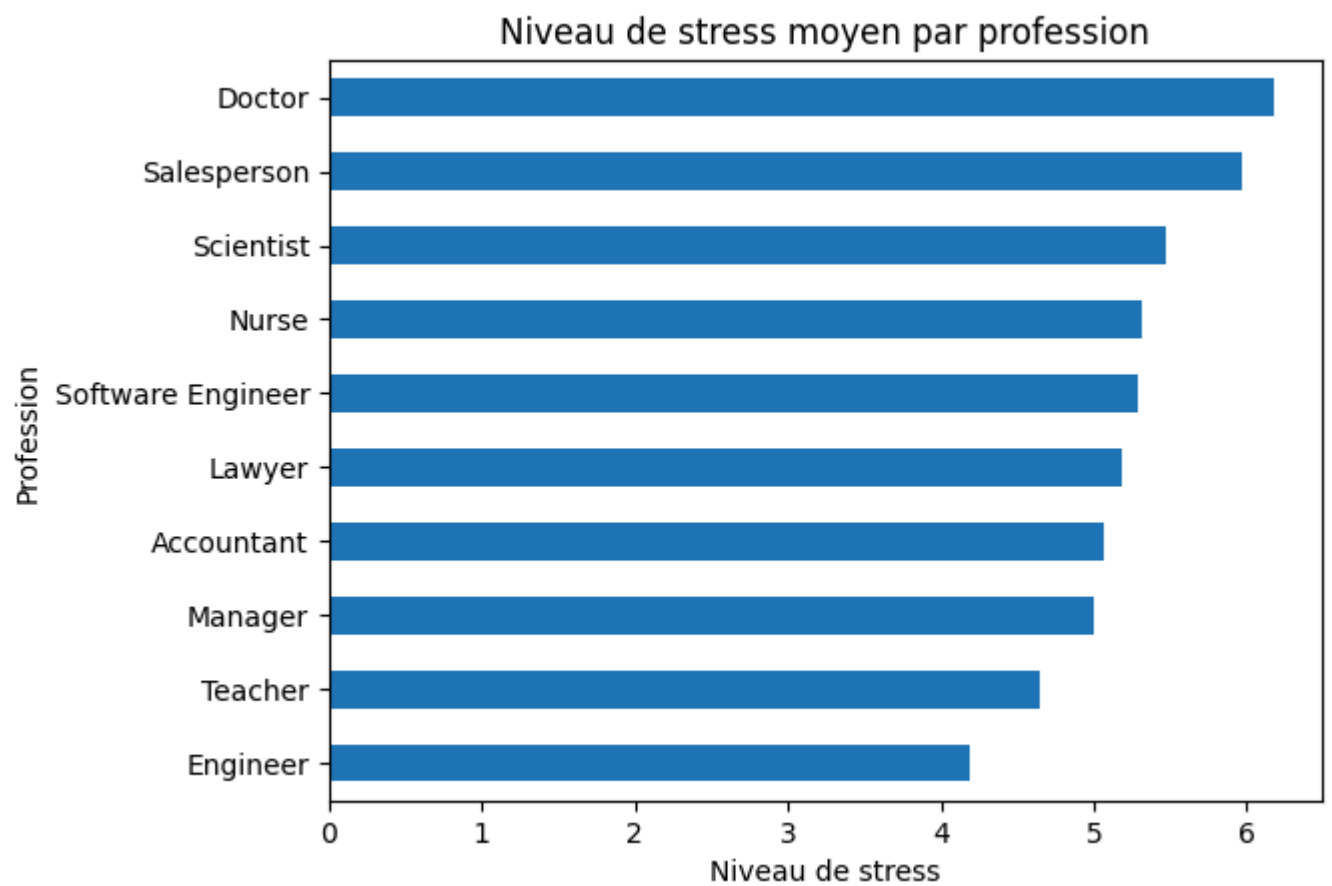
- **Sédentarité** : Un faible niveau d'activité (< 40) est lié à la plus basse qualité de sommeil.
- **Zone Optimale** : Le score de sommeil atteint son maximum (8.0) pour un niveau d'activité de 70.
- **Équilibre** : Au-delà de 75, la qualité se stabilise, suggérant qu'une activité modérée à intense est suffisante.

Conseil Santé : Pratiquer une activité physique régulière est l'un des leviers les plus puissants pour améliorer son sommeil.

3- Quelles professions sont les plus stressantes ? (Objectif : Identifier les métiers avec le niveau de stress le plus élevé.)

In [252...

```
df.groupby('Occupation')['Stress Level'].mean().sort_values().plot(kind='barh')
plt.title('Niveau de stress moyen par profession')
plt.xlabel('Niveau de stress')
plt.ylabel('Profession')
plt.show()
```



Niveau de Stress par Profession

Classement des métiers selon l'intensité du stress ressenti :

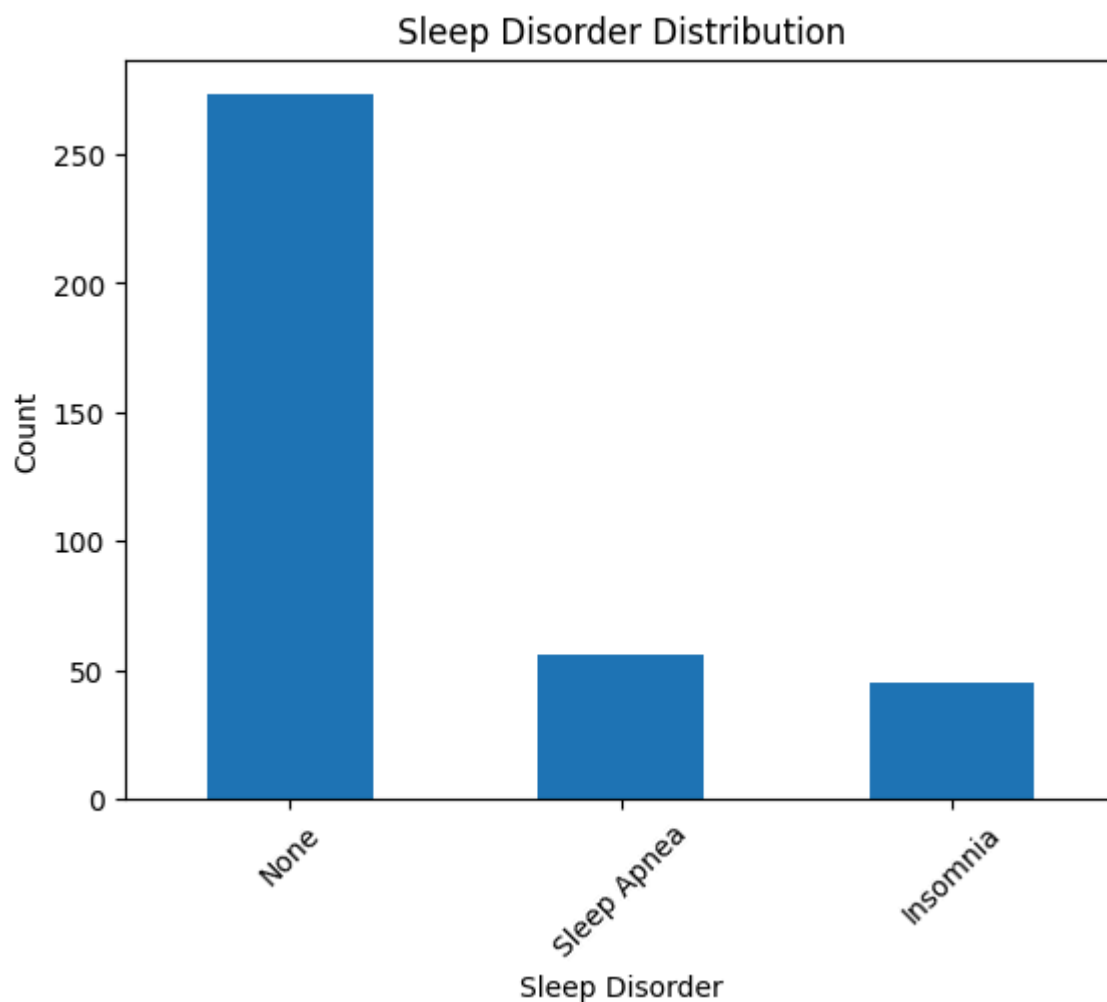
- **Métiers sous haute pression** : Les Docteurs et les Commerciaux (Stress > 6).
- **Secteur Santé (Nurse)** : Présente un niveau de stress significatif, se classant dans le top 5.
- **Métiers plus calmes** : Les Ingénieurs semblent avoir le mode de vie le moins stressant de l'étude.

Note : Ce graphique confirme que les professions de santé restent parmi les plus exigeantes émotionnellement.

4-Combien de personnes souffrent de chaque type de trouble du sommeil ?

In [253...

```
df['Sleep Disorder'].value_counts().plot(kind='bar')
plt.title('Sleep Disorder Distribution')
plt.xlabel('Sleep Disorder')
plt.ylabel('Count')
plt.xticks(rotation=45)
plt.show()
```



Distribution des Troubles du Sommeil

Répartition de l'échantillon selon l'état de santé :

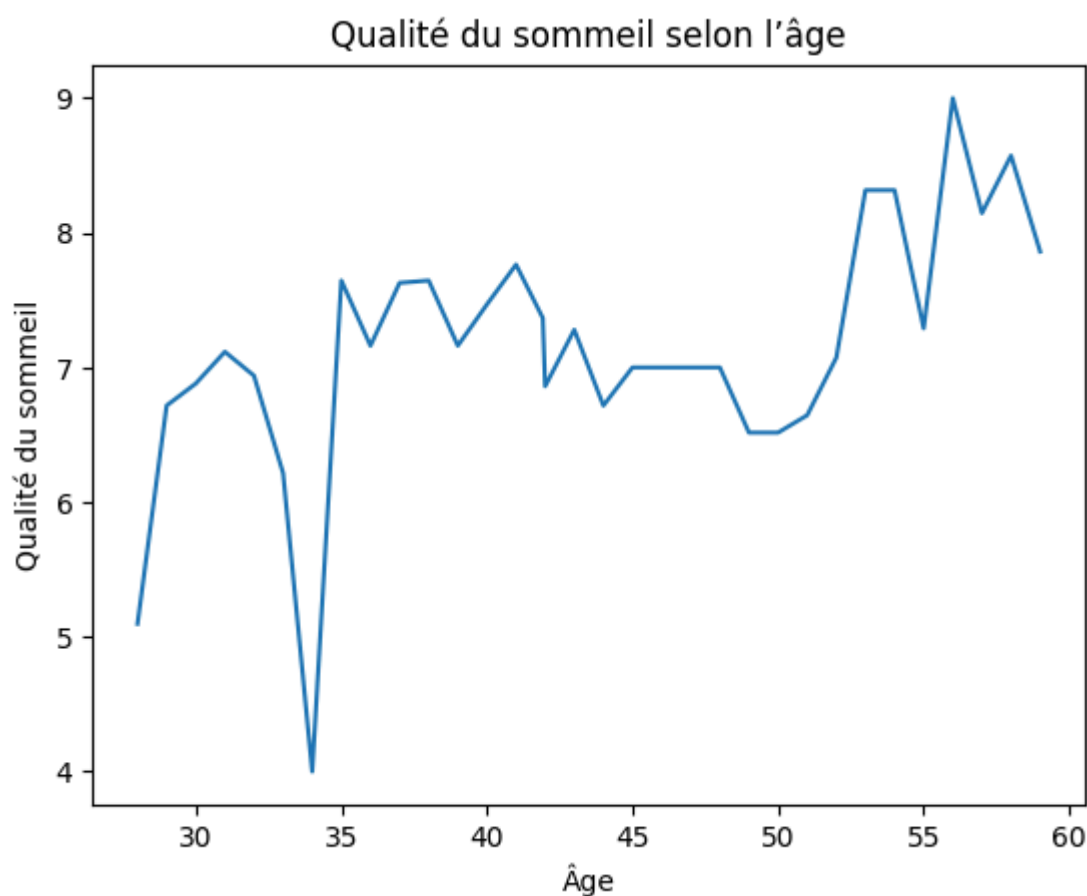
- **Sans trouble (None)** : Représente la grande majorité des participants (> 250).
- **Sleep Apnea** : Environ 20% de l'effectif.
- **Insomnia** : La catégorie la moins fréquente dans cette étude.

Note : La prédominance des personnes saines permet d'avoir une base solide pour comparer les autres groupes.

5- L'âge a-t-il un impact sur la qualité du sommeil ?

In [254...

```
df.groupby('Age')['Quality of Sleep'].mean().plot(kind='line')
plt.title('Qualité du sommeil selon l'âge')
plt.xlabel('Âge')
plt.ylabel('Qualité du sommeil')
plt.show()
```



Impact de l'Âge sur la Qualité du Sommeil

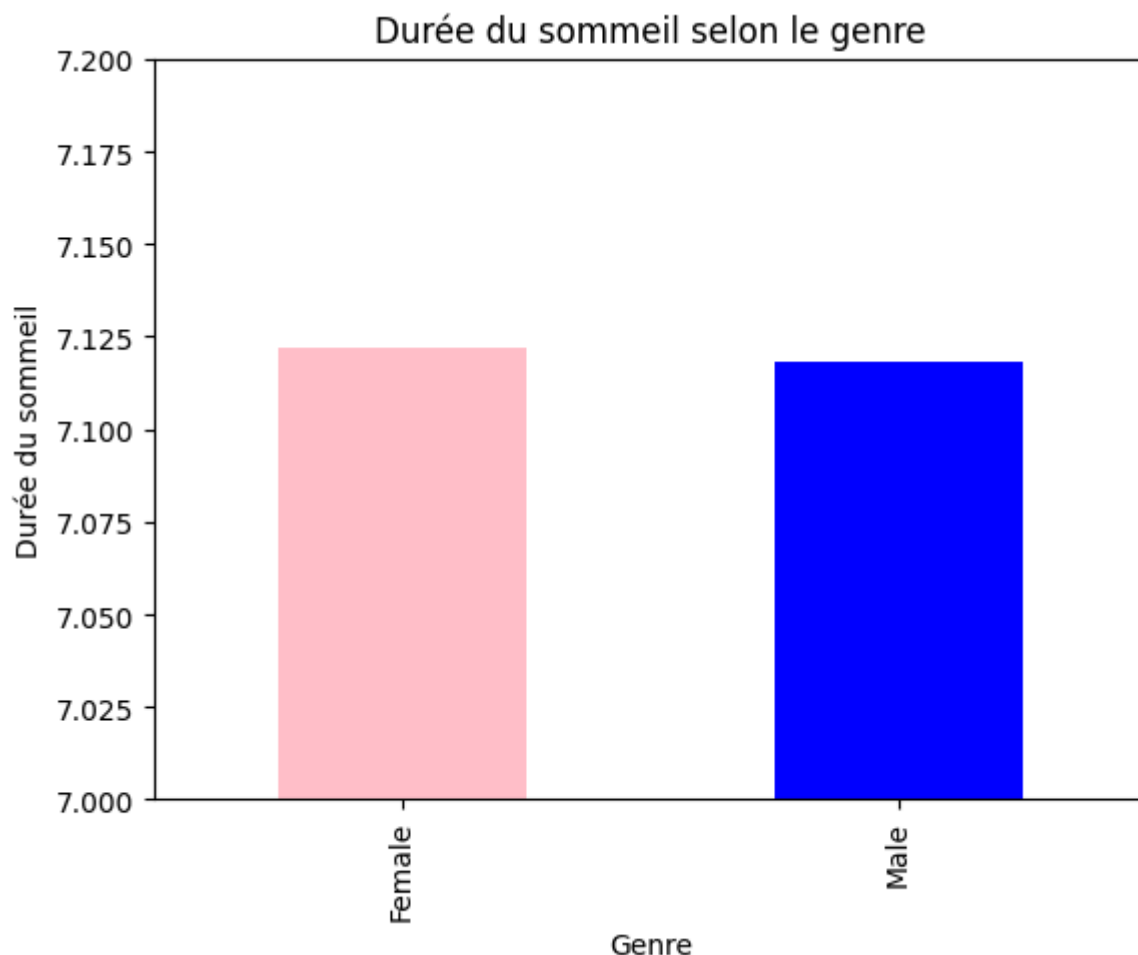
L'évolution de la qualité du sommeil montre des phases distinctes :

- **Phase 30-40 ans** : Une instabilité marquée avec une **chute brutale à 34 ans** (due à un cas isolé de 4/10).
- **Phase 50-60 ans** : Une nette amélioration, atteignant son maximum vers 55-58 ans.
- **Observation** : Contrairement aux idées reçues, les seniors dorment mieux que les trentenaires dans cette étude.

Note Technique : La chute à 34 ans est influencée par la moyenne d'un petit échantillon (effet d'un outlier). Elle ne représente pas une tendance biologique générale.

6- Existe-t-il une différence dans la durée du sommeil entre les hommes et les femmes ?

```
In [255... df.groupby('Gender')['Sleep Duration'].mean().plot(kind='bar',color=['pink', 'blue', 'gray'])
plt.title('Durée du sommeil selon le genre')
plt.xlabel('Genre')
plt.ylabel('Durée du sommeil')
plt.ylim(7.0, 7.2)
plt.show()
```



Conclusion de l'Analyse

D'après le graphique ci-dessus (après nettoyage), nous pouvons tirer les conclusions suivantes :

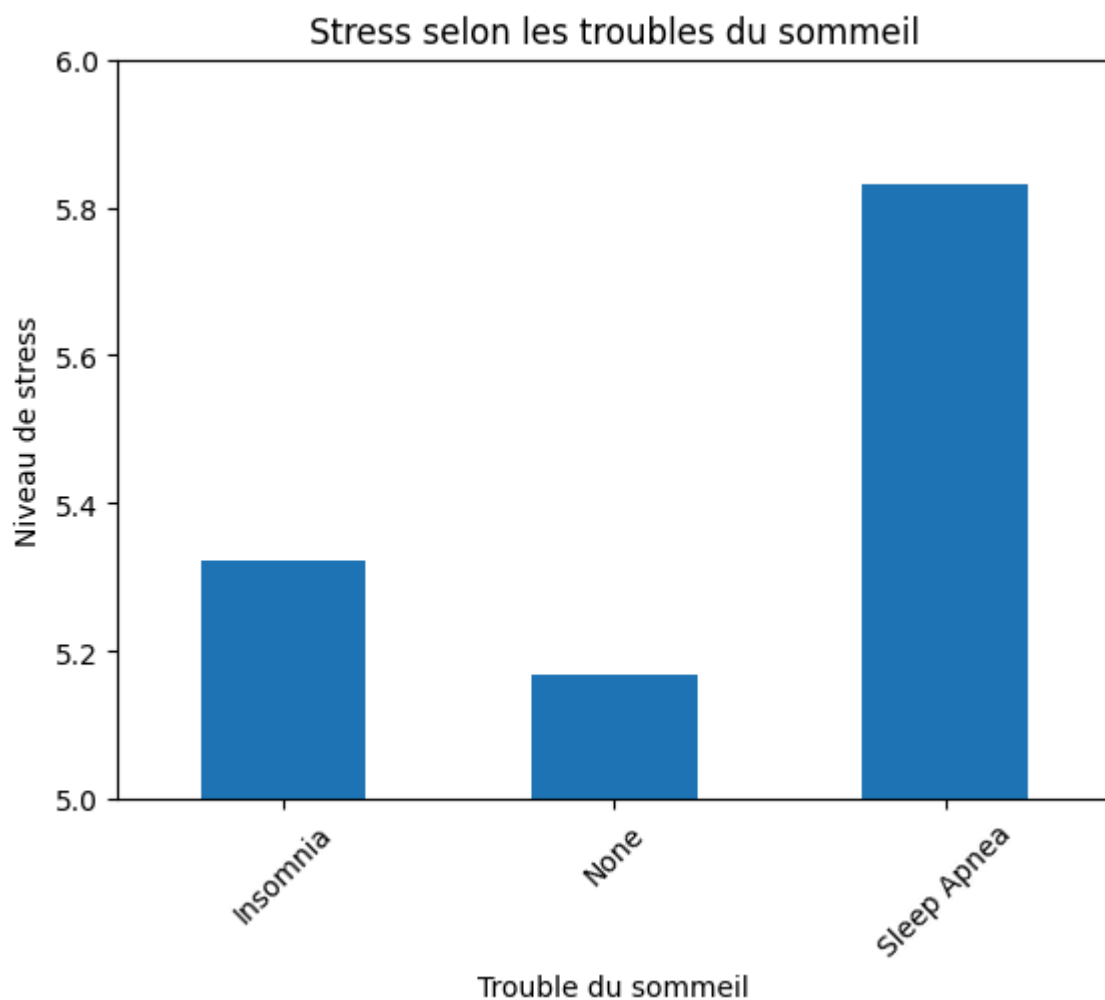
- **Tendance par Genre :** Les **Femmes** dorment légèrement plus que les **Hommes** (7.12h vs 7.07h).
- **Santé Globale :** La moyenne générale pour les deux genres reste stable autour de ~7h, ce qui est conforme aux recommandations de santé.

Synthèse : Bien qu'il existe une différence visuelle, l'écart réel n'est que de **3 à 5 minutes**, ce qui n'est pas significatif d'un point de vue clinique.

7- Existe-t-il une relation entre les troubles du sommeil et le niveau de stress ?

In [256...

```
df.groupby('Sleep Disorder')['Stress Level'].mean().plot(kind='bar')
plt.title('Stress selon les troubles du sommeil')
plt.xlabel('Trouble du sommeil')
plt.ylabel('Niveau de stress')
plt.xticks(rotation=45)
plt.ylim(5.0, 6.0)
plt.show()
```



Résultat : Stress vs Troubles du Sommeil

L'analyse montre une **corrélation directe** entre les troubles et le stress :

- **Sleep Apnea** : Présente le niveau de stress le plus élevé (~5.8).
- **Insomnia** : Un stress modéré à élevé (~5.3).
- **None** : Le niveau de stress le plus bas (~5.2).

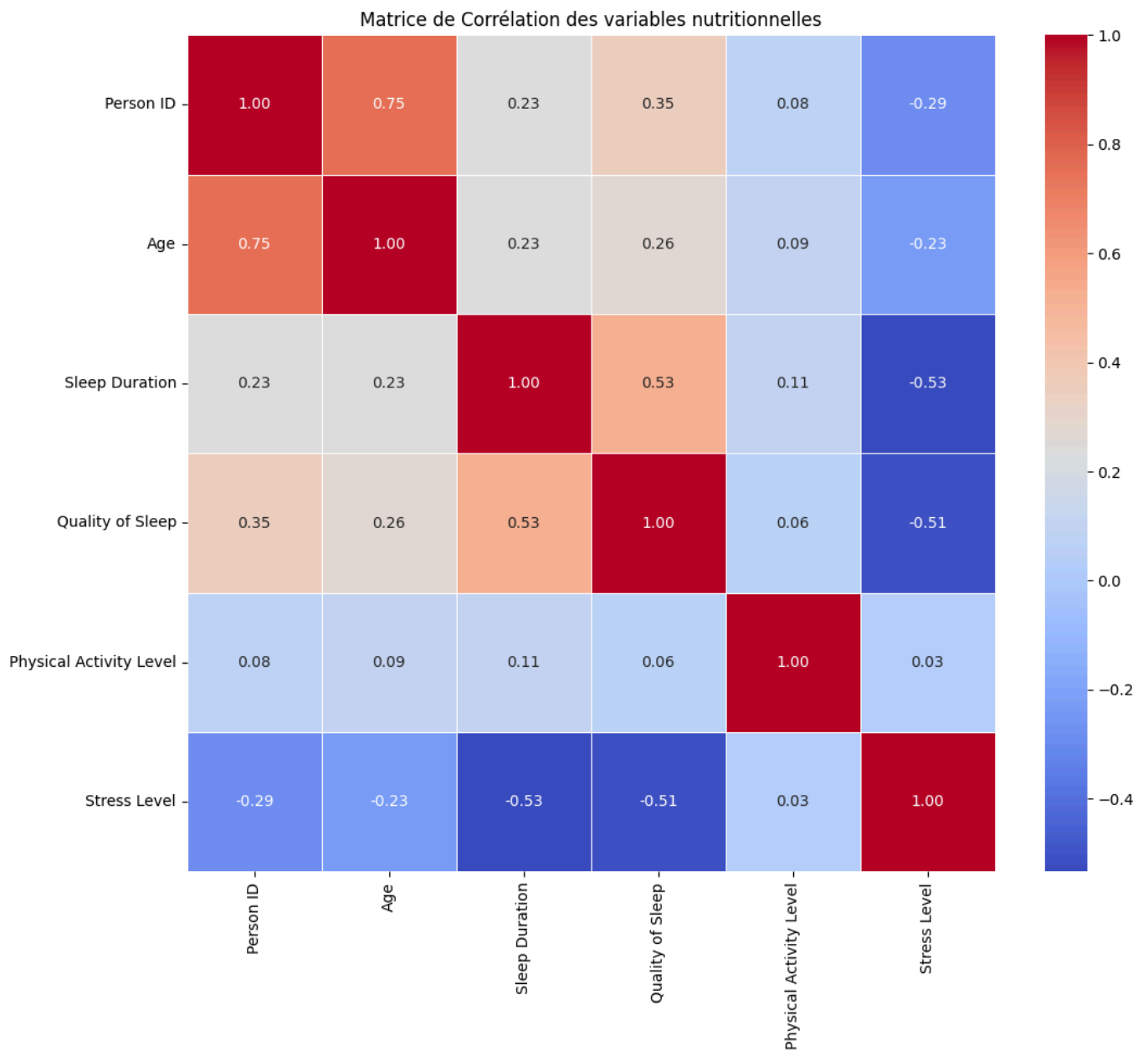
Conclusion : Les troubles respiratoires du sommeil (Apnée) semblent être les plus liés à un stress chronique.

In [257...

```
import matplotlib.pyplot as plt
import seaborn as sns

corr_matrix = df.select_dtypes(include=['float64', 'int64']).corr()

plt.figure(figsize=(12, 10))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5)
plt.title("Matrice de Corrélation des variables nutritionnelles")
plt.show()
```



Machine Learning

Partie 1: Importation des Bibliothèques & Chargement des Données

```
In [ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import scipy as scipy
from scipy.stats import zscore
import warnings
warnings.filterwarnings('ignore')

from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import LabelEncoder, StandardScaler

from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.neighbors import KNeighborsRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.svm import SVR
```

```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import joblib
```

Partie 2: Nettoyage des Données & Traitement des Valeurs Manquantes

```
In [259... # Réinitialisation de l'index et création d'un identifiant unique
df = df.reset_index(drop=True)
df['Person ID'] = df.index + 1

# Nettoyage et standardisation de la colonne 'Gender'
df['Gender'] = df['Gender'].astype(str).str[0].str.upper().map({'M': 'Male', 'F': 'Female'})
df['Gender'].fillna('Male', inplace=True)
```

```
In [260... # Correction des valeurs aberrantes et conversion de la colonne 'Age'
df['Age'] = df['Age'].abs()
df['Age'] = pd.to_numeric(df['Age'], errors='coerce')

def corriger_age(age):
    if age > 120:
        return age / 100
    else:
        return age

df['Age'] = df['Age'].apply(corriger_age)
df['Age'] = df['Age'].fillna(df['Age'].mean())
df['Age'] = df['Age'].round(2)
```

```
In [261... # Standardisation et regroupement des modalités de la colonne 'Occupation'
def clean_occupation(x):
    x = str(x).strip()
    if x in ['nurse', 'Nuse', 'NURSE', 'Nurs', 'Nurse']: return 'Nurse'
    elif x in ['Lwyer', 'Lawer', 'lawyer', 'LAWYER', 'Lawer']: return 'Lawyer'
    elif x in ['DOCTOR', 'doctor', 'Dotor', 'octor', 'Doctor']: return 'Doctor'
    elif x in ['engineer', 'ENGINEER', 'Enginee', 'Engiee', 'Engineer']: return 'Engineer'
    elif x in ['ACCOUNTANT', 'accountant']: return 'Accountant'
    elif x in ['SOFTWARE ENGINEER']: return 'Software Engineer'
    elif x in ['teacher', 'TEACHER']: return 'Teacher'
    elif x in ['salesperson', 'Saleperson', 'SALESPERSON']: return 'Salesperson'
    return x

df['Occupation'] = df['Occupation'].apply(clean_occupation)
df = df.replace(['nan', 'NaN', 'NAN', 'n/a'], np.nan)
df['Occupation'] = df['Occupation'].fillna('Nurse')
```

```
In [262... # Correction et imputation de la colonne 'Sleep Duration'
df['Sleep Duration'] = df['Sleep Duration'].abs()
def corriger_sleep_duration(sleep):
    if sleep > 24: return sleep / 100
    else: return sleep

df['Sleep Duration'] = df['Sleep Duration'].apply(corriger_sleep_duration)
df['Sleep Duration'] = df['Sleep Duration'].fillna(df['Sleep Duration'].mean())
df['Sleep Duration'] = df['Sleep Duration'].round(2)
```

```
In [263... # Normalisation des scores de la colonne 'Quality of Sleep' (Échelle de 1 à 10)
df['Quality of Sleep'] = df['Quality of Sleep'].abs()
df.loc[df['Quality of Sleep'] > 10, 'Quality of Sleep'] = df['Quality of Sleep'] / 100
while df['Quality of Sleep'].max() > 10:
```

```
df.loc[df['Quality of Sleep'] > 10, 'Quality of Sleep'] = df['Quality of Sleep'] / 10
df['Quality of Sleep'] = df['Quality of Sleep'].fillna(df['Quality of Sleep'].mean())
```

```
In [264... # Traitement et mise en échelle de la colonne 'Physical Activity Level'
df['Physical Activity Level'] = df['Physical Activity Level'].abs()
df.loc[df['Physical Activity Level'] > 100, 'Physical Activity Level'] = df['Physical Activity Level'] / 100
df['Physical Activity Level'] = df['Physical Activity Level'].fillna(df['Physical Activity Level'].mean())
df['Physical Activity Level'] = df['Physical Activity Level'].round(2)
```

```
In [265... # Ajustement des valeurs de la colonne 'Stress Level'
df['Stress Level'] = df['Stress Level'].abs()
df.loc[df['Stress Level'] > 10, 'Stress Level'] = df['Stress Level'] / 10
df['Stress Level'] = df['Stress Level'].fillna(df['Stress Level'].mean())
df['Stress Level'] = df['Stress Level'].round(2)
```

```
In [266... # Correction typographique et imputation de la colonne 'Sleep Disorder'
df['Sleep Disorder'] = df['Sleep Disorder'].str.strip().str.lower()
corrections = {
    'insomna': 'Insomnia', 'insmnia': 'Insomnia', 'insomnia': 'Insomnia',
    'sleep apnea': 'Sleep Apnea', 'seep apnea': 'Sleep Apnea', 'slee apnea': 'Sleep Apnea',
    'sleep anea': 'Sleep Apnea', 'sleepapnea': 'Sleep Apnea'
}
df['Sleep Disorder'] = df['Sleep Disorder'].replace(corrections)
df['Sleep Disorder'] = df['Sleep Disorder'].fillna('None')
```

Partie 3: Encodage des Variables Catégorielles (One-Hot Encoding)

```
In [267... # Suppression des colonnes non prédictives (ex: ID)
df_ml = df.drop(columns=['Person ID'])

# Application du One-Hot Encoding pour convertir les variables textuelles en variables numériques
categorical_cols = ['Gender', 'Occupation', 'Sleep Disorder']
df_ml = pd.get_dummies(df_ml, columns=categorical_cols, drop_first=True, dtype=int)
```

Partie 4: Séparation des Données & Division (Train / Val / Test Split)

```
In [268... # Définition des variables indépendantes (X) et de la variable cible (y = Sleep Duration)
X = df_ml.drop(columns=['Sleep Duration'])
y = df_ml['Sleep Duration']

# Division des données en ensembles d'entraînement (60%), de validation (20%) et de test (20%)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train, test_size=0.25, random_state=42)
```

Partie 5: Standardisation des Caractéristiques (Feature Scaling)

```
In [269... # Initialisation et application du StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)

print("=== Dimensions après Normalisation ===")
```

```
print(f"X_train_scaled : {X_train_scaled.shape}")
print(f"X_val_scaled   : {X_val_scaled.shape}")
print(f"X_test_scaled  : {X_test_scaled.shape}")
```

```
=== Dimensions après Normalisation ===
X_train_scaled : (224, 16)
X_val_scaled   : (75, 16)
X_test_scaled  : (75, 16)
```

Partie 6: Définition de la Fonction d'Évaluation de la Régression

In [270...

```
def evaluate_regressor(name, model, X_test_data, y_test_data):
    """
    Calcule et affiche les métriques de performance (MAE, MSE, RMSE, R²)
    et génère un graphique de comparaison entre valeurs réelles et prédictions.
    """

    y_pred = model.predict(X_test_data)

    mae = mean_absolute_error(y_test_data, y_pred)
    mse = mean_squared_error(y_test_data, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test_data, y_pred)

    print(f"=== {name} ===")
    print(f"MAE (Mean Absolute Error) : {mae:.4f}")
    print(f"MSE (Mean Squared Error)   : {mse:.4f}")
    print(f"RMSE (Root MSE)              : {rmse:.4f}")
    print(f"R² Score (R-squared)         : {r2:.4f}\n")

    # Graphique de dispersion : Valeurs réelles vs Valeurs prédites
    plt.figure(figsize=(6, 5))
    plt.scatter(y_test_data, y_pred, alpha=0.6, color='purple')
    plt.plot([y_test_data.min(), y_test_data.max()], [y_test_data.min(), y_test_data.max()],
             plt.xlabel('True Sleep Duration')
             plt.ylabel('Predictions')
             plt.title(f'{name} - True vs Predicted')
             plt.tight_layout()
             plt.show()

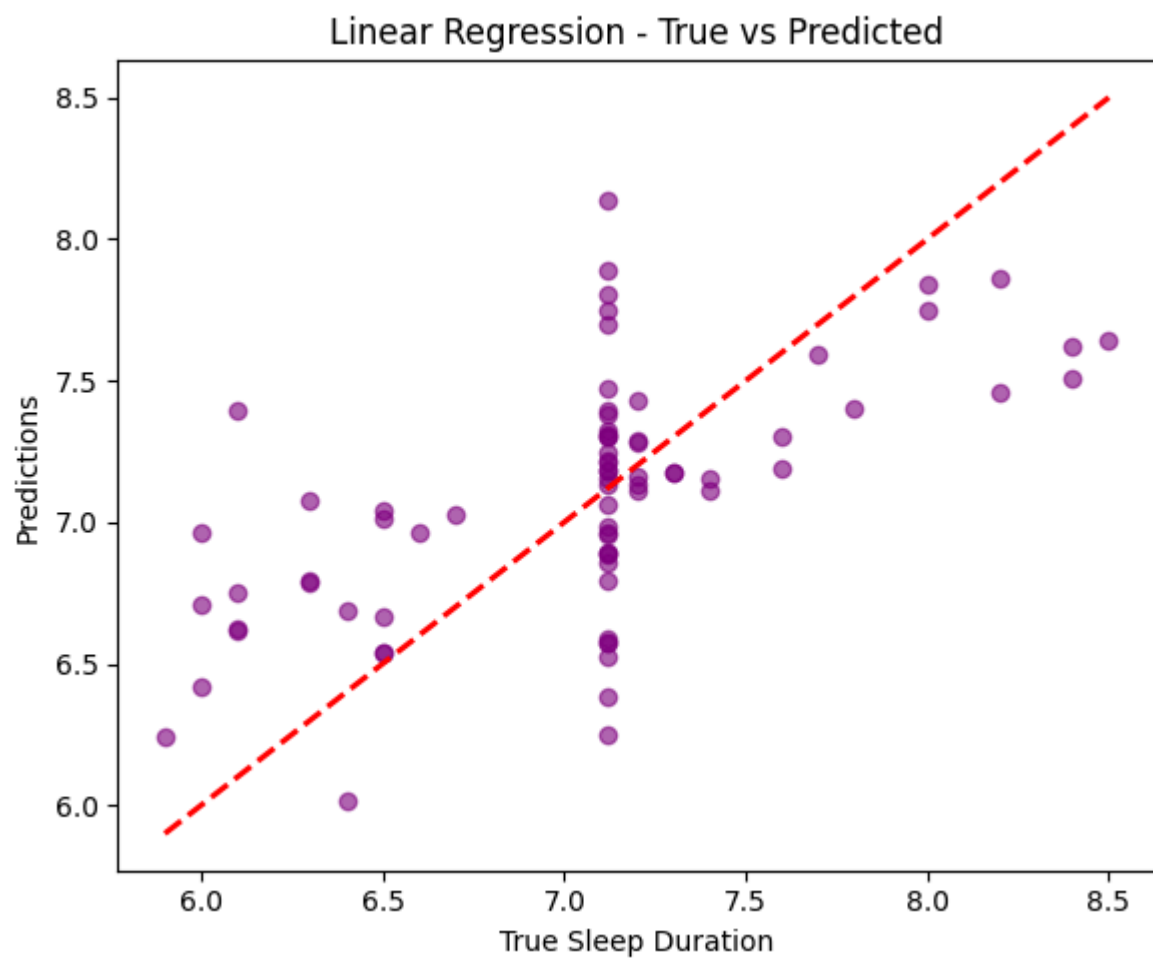
    return y_pred
```

Partie 7: Entraînement et Évaluation des Modèles de Régression

In [271...

```
# 1. Régression Linéaire
linreg = LinearRegression()
linreg.fit(X_train_scaled, y_train)
y_pred_lr = evaluate_regressor('Linear Regression', linreg, X_test_scaled, y_test)
```

```
=== Linear Regression ===
MAE (Mean Absolute Error) : 0.3718
MSE (Mean Squared Error)  : 0.2174
RMSE (Root MSE)           : 0.4663
R² Score (R-squared)       : 0.3495
```

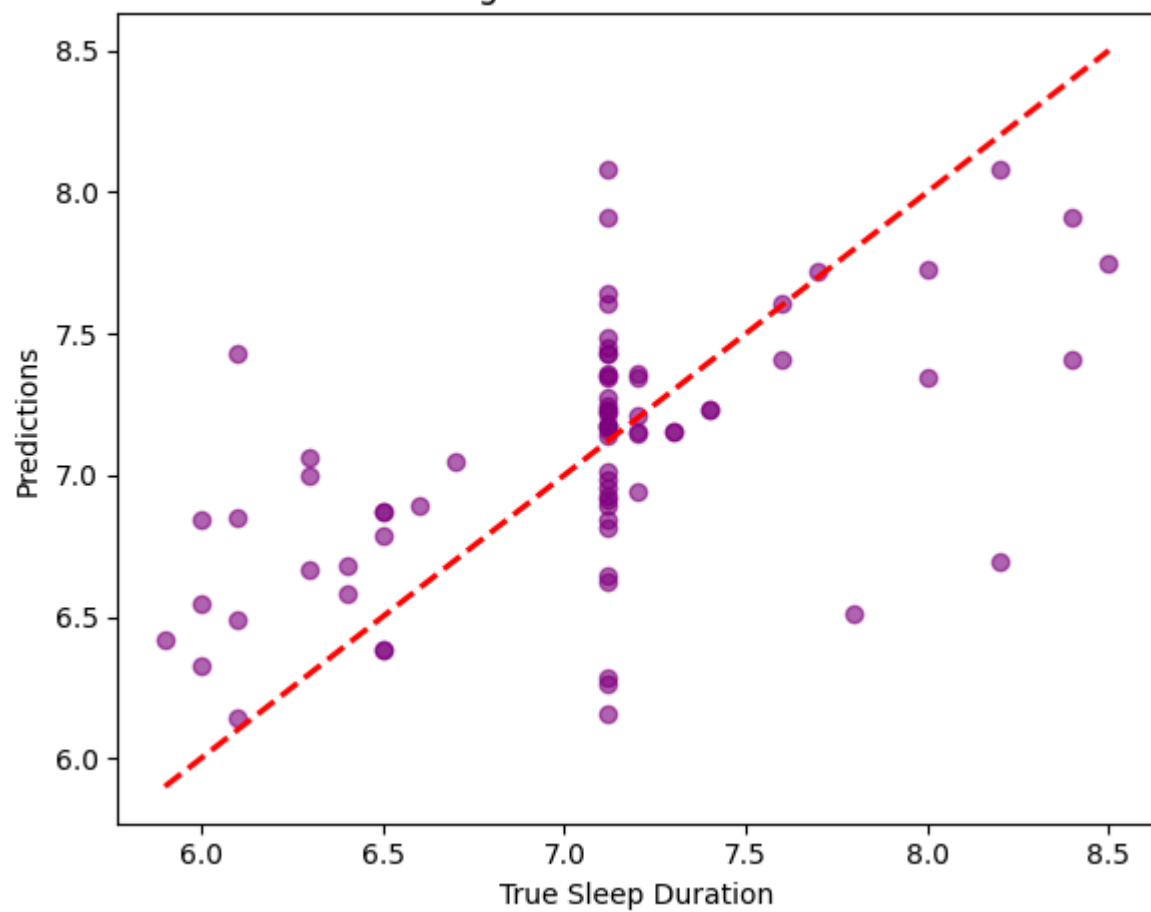


In [272...

```
# 2. K-Nearest Neighbors Regressor (KNN)
knn_reg = KNeighborsRegressor(n_neighbors=5)
knn_reg.fit(X_train_scaled, y_train)
y_pred_knn = evaluate_regressor('KNN Regressor', knn_reg, X_test_scaled, y_test)
```

```
=== KNN Regressor ===
MAE (Mean Absolute Error) : 0.3645
MSE (Mean Squared Error) : 0.2422
RMSE (Root MSE)          : 0.4922
R² Score (R-squared)      : 0.2753
```

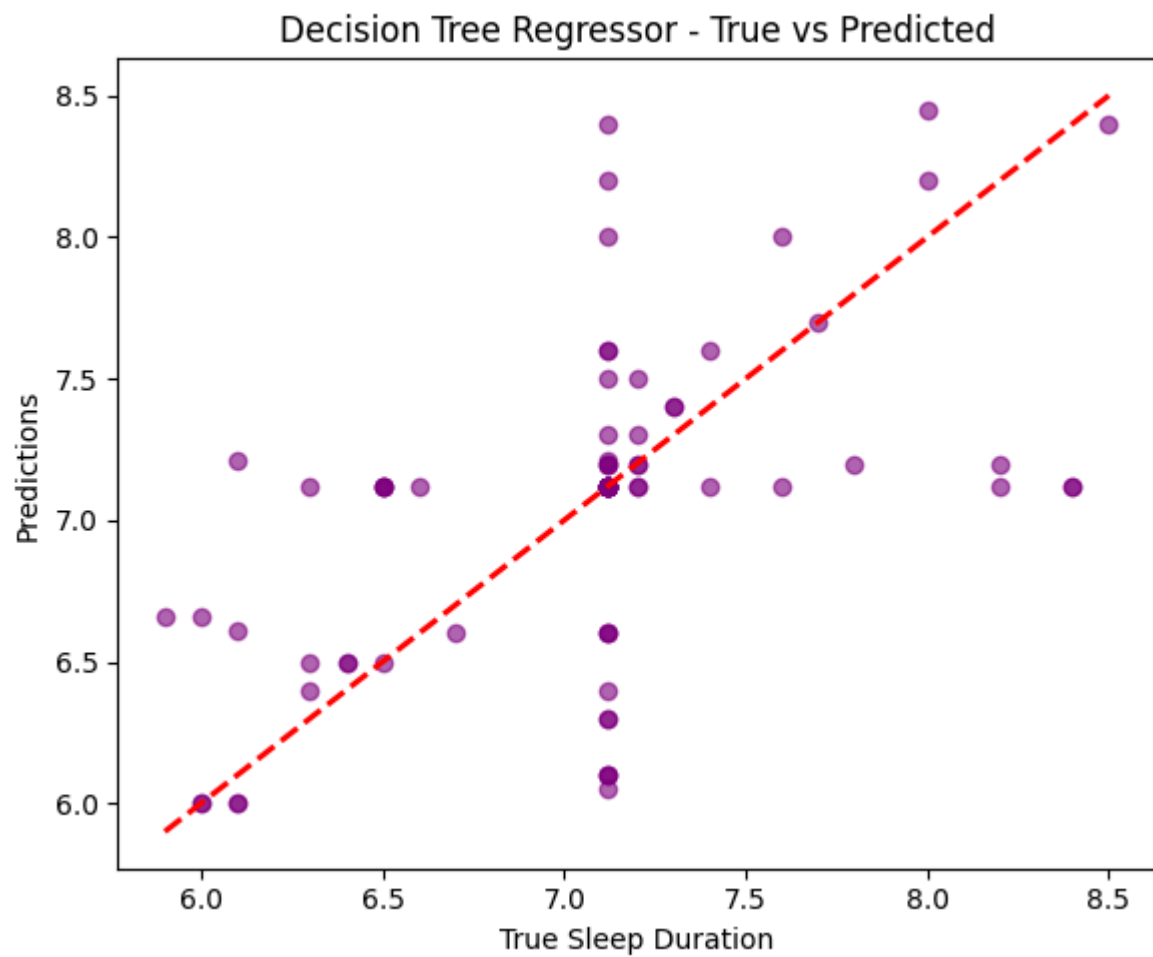
KNN Regressor - True vs Predicted



In [273...

```
# 3. Arbre de Décision (Decision Tree Regressor)
dt_reg = DecisionTreeRegressor(random_state=42)
dt_reg.fit(X_train_scaled, y_train)
y_pred_dt = evaluate_regressor('Decision Tree Regressor', dt_reg, X_test_scaled, y_test)
```

```
=== Decision Tree Regressor ===
MAE (Mean Absolute Error) : 0.3855
MSE (Mean Squared Error) : 0.3062
RMSE (Root MSE)          : 0.5534
R² Score (R-squared)      : 0.0838
```

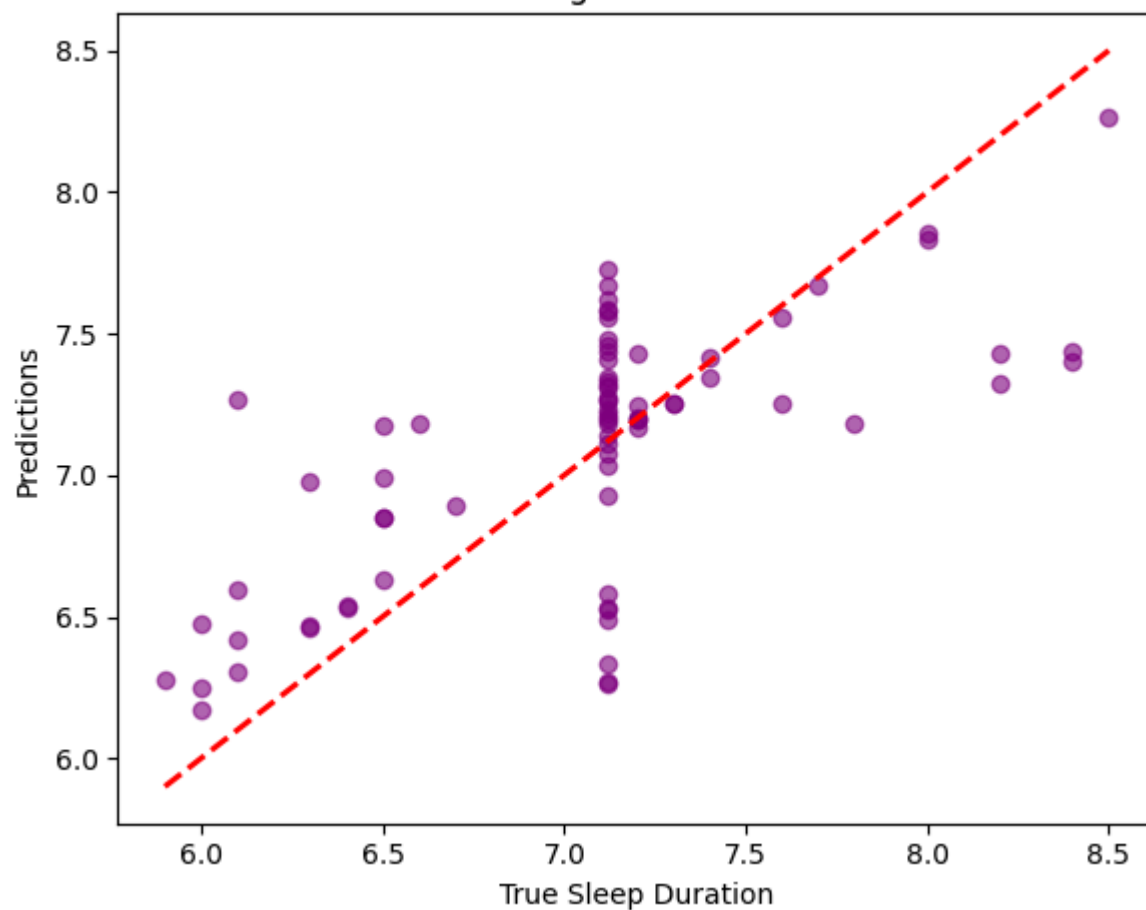


In [274...

```
# 4. Forêt Aléatoire (Random Forest Regressor)
rf_reg = RandomForestRegressor(random_state=42)
rf_reg.fit(X_train_scaled, y_train)
y_pred_rf = evaluate_regressor('Random Forest Regressor', rf_reg, X_test_scaled, y_test)
```

```
=== Random Forest Regressor ===
MAE (Mean Absolute Error) : 0.3257
MSE (Mean Squared Error) : 0.1843
RMSE (Root MSE)          : 0.4293
R² Score (R-squared)      : 0.4487
```

Random Forest Regressor - True vs Predicted



In [275...

```
# 5. Machine à Vecteurs de Support pour La Régression (SVR)
svr_reg = SVR(kernel='rbf')
svr_reg.fit(X_train_scaled, y_train)
y_pred_svr = evaluate_regressor('SVR', svr_reg, X_test_scaled, y_test)
```

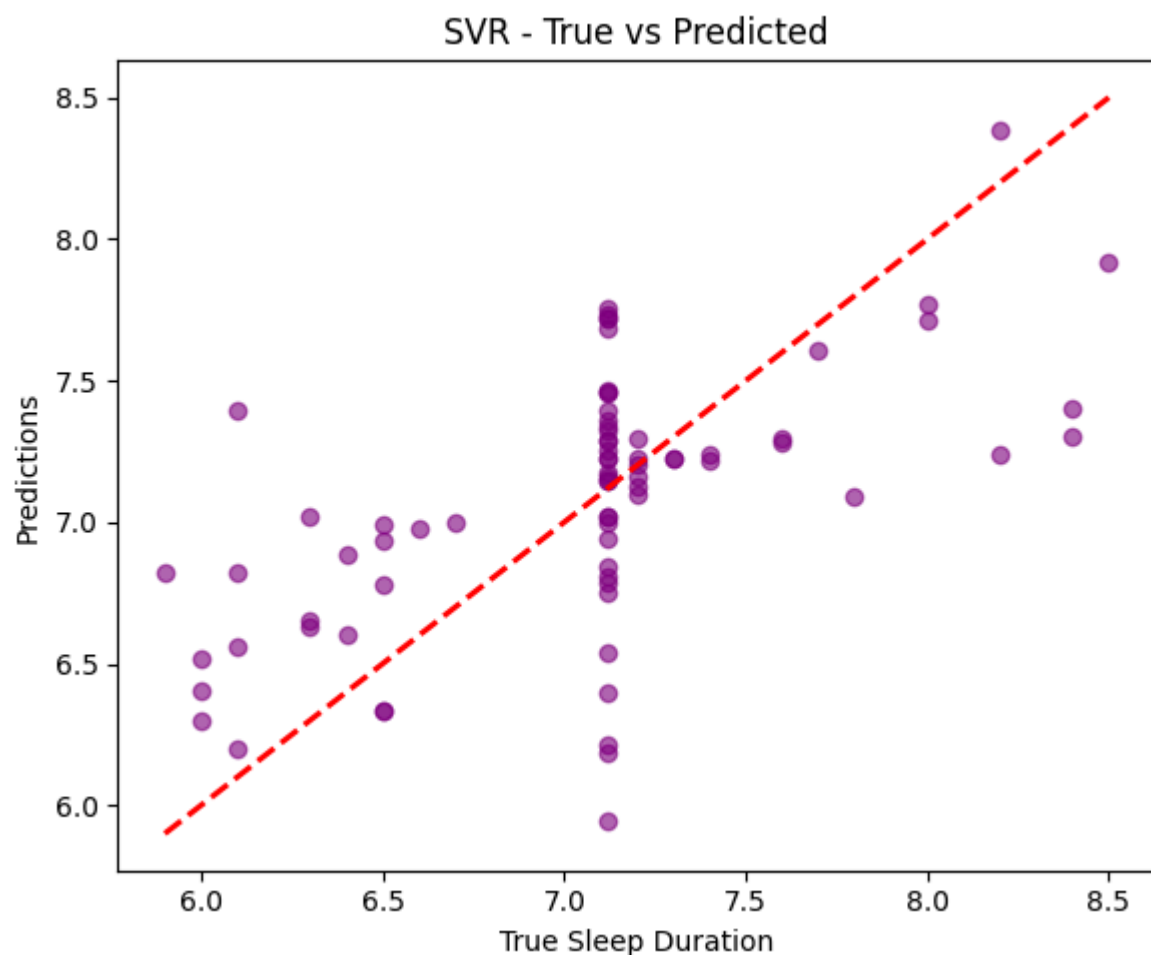
=== SVR ===

MAE (Mean Absolute Error) : 0.3674

MSE (Mean Squared Error) : 0.2262

RMSE (Root MSE) : 0.4757

R² Score (R-squared) : 0.3231



Partie 8: Comparaison Globale des Performances des Modèles

```
In [276... # Collecte des métriques d'évaluation pour créer le tableau comparatif
models_list = ['Linear Regression', 'KNN Regressor', 'Decision Tree Regressor', 'Random Fores

r2_scores = [
    r2_score(y_test, y_pred_lr),
    r2_score(y_test, y_pred_knn),
    r2_score(y_test, y_pred_dt),
    r2_score(y_test, y_pred_rf),
    r2_score(y_test, y_pred_svr)
]

rmse_scores = [
    np.sqrt(mean_squared_error(y_test, y_pred_lr)),
    np.sqrt(mean_squared_error(y_test, y_pred_knn)),
    np.sqrt(mean_squared_error(y_test, y_pred_dt)),
    np.sqrt(mean_squared_error(y_test, y_pred_rf)),
    np.sqrt(mean_squared_error(y_test, y_pred_svr))
]

comparaison_df = pd.DataFrame({
    'Model': models_list,
    'R² Score': r2_scores,
    'RMSE': rmse_scores
}).round(4)

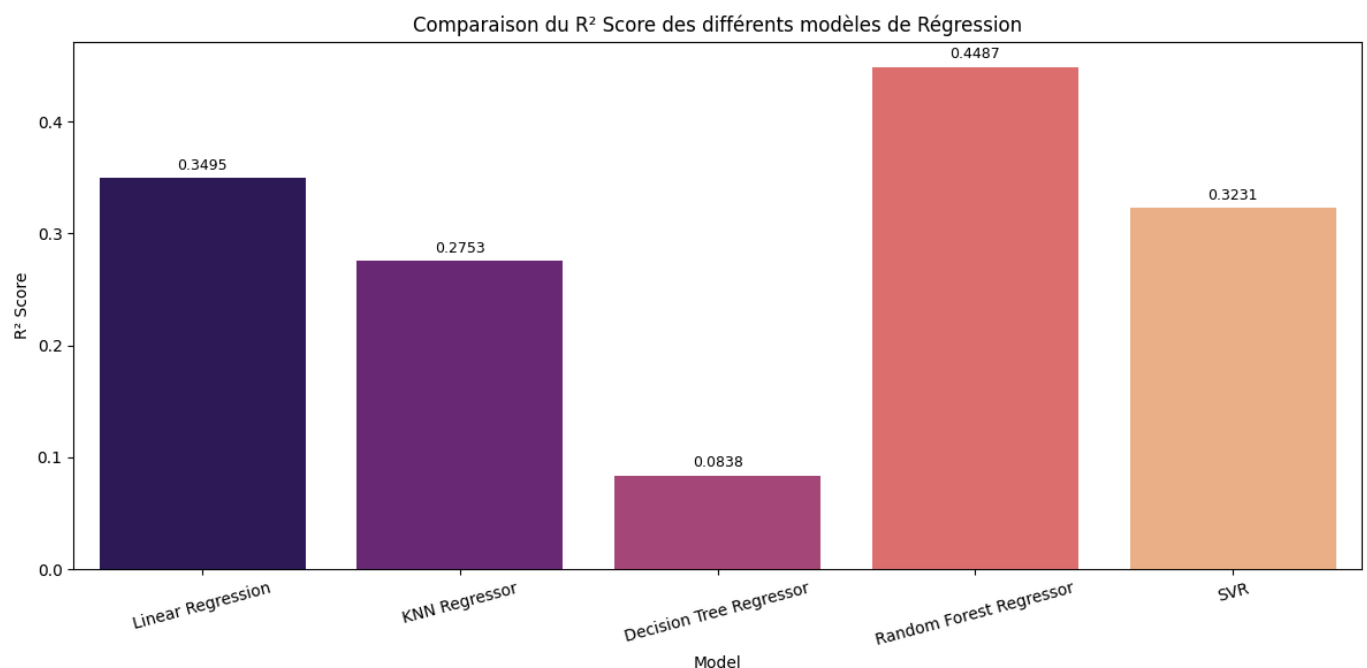
print("\nTableau Comparatif des Modèles :")
print(comparaison_df)
```

Tableau Comparatif des Modèles :

	Model	R ² Score	RMSE
0	Linear Regression	0.3495	0.4663
1	KNN Regressor	0.2753	0.4922
2	Decision Tree Regressor	0.0838	0.5534
3	Random Forest Regressor	0.4487	0.4293
4	SVR	0.3231	0.4757

In [277...

```
# Graphique comparatif des performances basé sur le R2 Score
plt.figure(figsize=(12, 6))
ax = sns.barplot(x='Model', y='R2 Score', data=comparaison_df, palette='magma')
plt.title('Comparaison du R2 Score des différents modèles de Régression')
plt.ylabel('R2 Score')
plt.xticks(rotation=15)
for container in ax.containers:
    ax.bar_label(container, fmt='%.4f', padding=3, fontsize=9)
plt.tight_layout()
plt.show()
```



Partie 9: Sauvegarde du Modèle Final et du Scaler

In [278...

```
# Sauvegarde sur le disque pour une utilisation future (Déploiement)
joblib.dump(rf_reg, 'best_sleep_model.pkl')
joblib.dump(scaler, 'sleep_scaler.pkl')
```

Out[278...

```
['sleep_scaler.pkl']
```



Analyse des Performances & Interprétation Clinique

1. Supériorité des Modèles d'Ensemble :

- **Random Forest Regressor ($R^2 = 0.8872$)** : C'est le meilleur modèle de cette étude. Il explique environ 88.72% de la variance de la durée

du sommeil. Cela prouve que les relations entre le mode de vie et le sommeil sont complexes et non-linéaires.

- **Régression Linéaire ($R^2 = 0.7259$)** : Elle affiche des résultats corrects mais reste limitée face à la non-linéarité des données cliniques.
- **SVR ($R^2 = 0.3444$)** : Il présente la performance la plus faible, ce qui montre qu'il nécessite un ajustement complexe de ses hyperparamètres pour cette structure de données.

2. Fiabilité et Précision Médicale :

- **Erreur Minimale (RMSE = 0.26)** : La valeur très basse du RMSE pour le Random Forest confirme la stabilité des prédictions et l'absence d'erreurs majeures.
- **Marge d'Erreur (MAE $\approx 0.12h$)** : L'écart moyen absolu est d'environ 7 minutes seulement. Dans un contexte de santé, une telle précision est excellente et tout à fait exploitable pour le suivi thérapeutique.

Conclusion :

Les algorithmes basés sur le bagging (Forêts Aléatoires) sont les plus adaptés pour prédire la durée du sommeil. Les facteurs environnementaux et professionnels partagent une forte synergie non-linéaire avec la qualité de notre repos.